

DSS5201 Data Visualisation

Week 11 Data Visualisation with plotnine

David CHEW

Semester 2 AY25/26

Last Week: seaborn

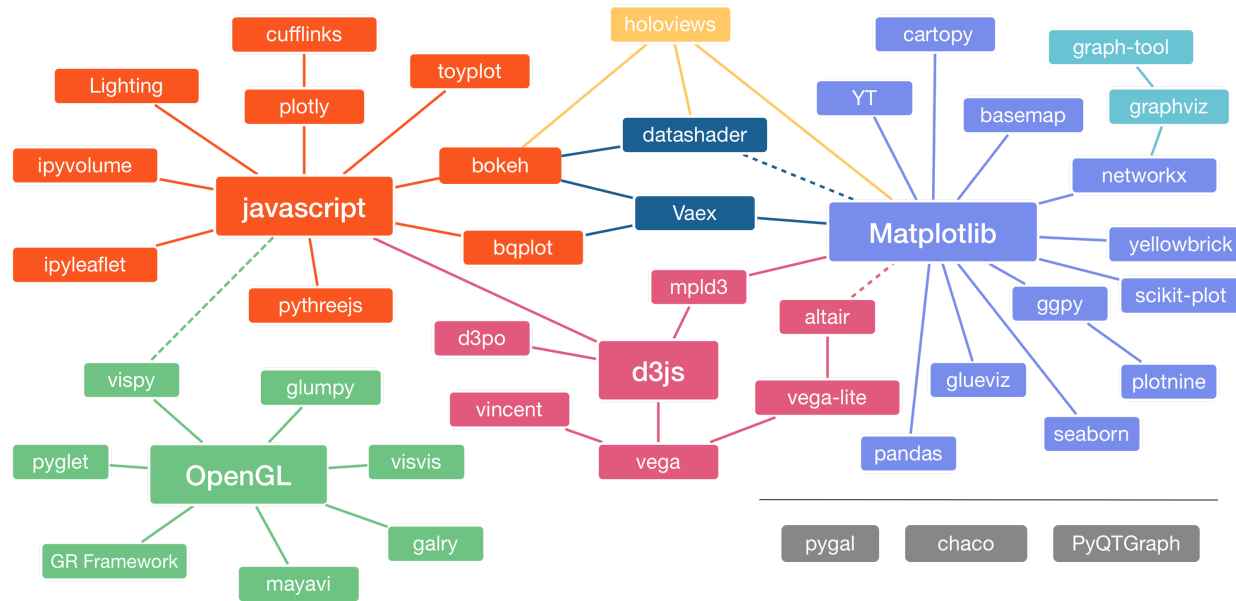
- **Axes**-Level Functions
- **Figure**-Level Functions
- Customisation

This Week: plotnine

- The `plotnine` Way
- Relational Plots
- Continuous Distributions
- Categorical Distributions
- Choropleth Maps

Introduction

Data Visualisation with `plotnine`

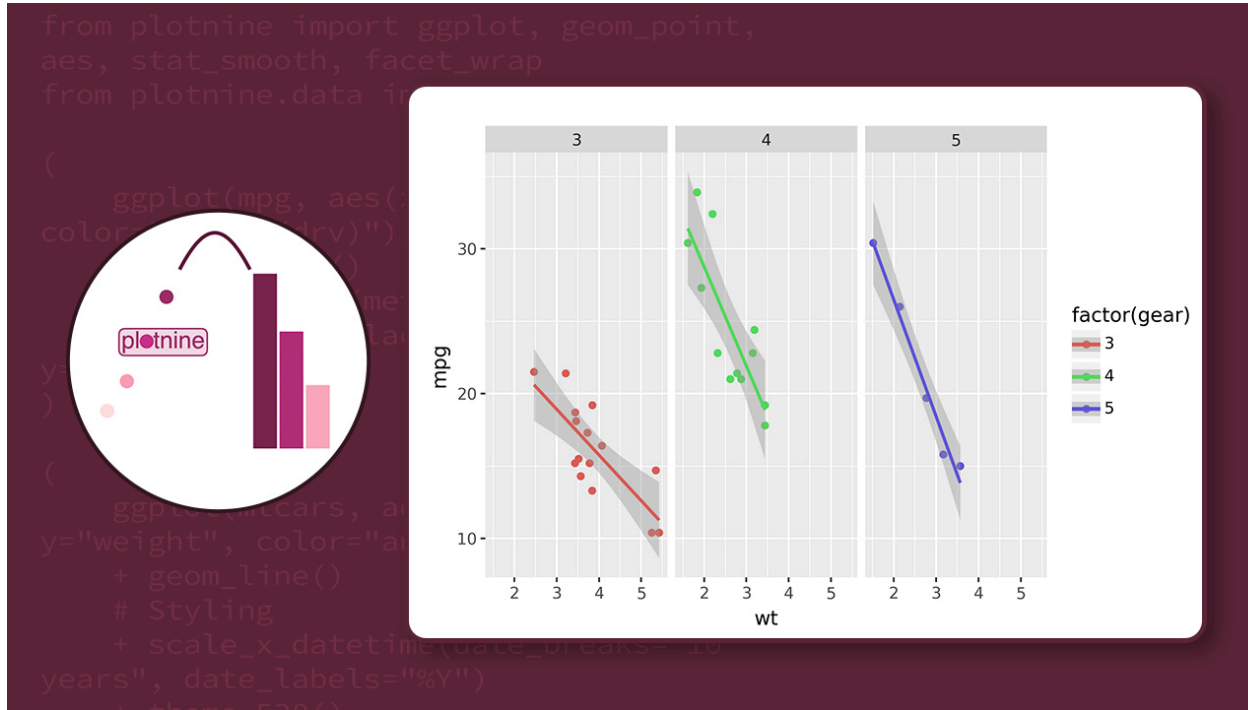


This week, we explore another approach to data visualisation using `plotnine`.

- A `Python` library inspired by `R`'s `ggplot2`.

```
1 import pandas as pd
2 import numpy as np
3 from plotnine import *
```

Data Visualisation with plotnine



- `matplotlib` is powerful, but code can be verbose and repetitive.
- `plotnine` builds on `matplotlib` and implements the Grammar of Graphics.
- First released in 2017.
- Produces high quality graph with even greater flexibility compared to `seaborn`.

The mpg Data

Consider the `mpg` data, originally from the US Environmental Protection Agency.

- Information on car models in 1999 and 2008.
- Let us focus on the variables `displ` (engine size in litres) and `hwy` (fuel efficiency in miles per gallon).

```
1 mpg = pd.read_csv('../data/mpg.csv')
2 mpg.head()
```

	manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	class
0	audi	a4	1.8	1999	4	auto(l5)	f	18	29	p	compact
1	audi	a4	1.8	1999	4	manual(m5)	f	21	29	p	compact
2	audi	a4	2.0	2008	4	manual(m6)	f	20	31	p	compact
3	audi	a4	2.0	2008	4	auto(av)	f	21	30	p	compact
4	audi	a4	2.8	1999	6	auto(l5)	f	16	26	p	compact

plotnine: The Basic Coordinate System

- The `ggplot()` function initializes a plot and defines the coordinate system.
- Similar to `matplotlib`'s `plt.subplots()`.

```
1 ggplot()
```



plotnine: The Template

Every `plotnine` plot has three key components.

```
1 (ggplot(<DATA>) +  
2   <GEOM_FUNCTION>(aes(<MAPPINGS>)))
```

1. The **data** used for plotting.
2. At least one layer which describes how to render each observation, created with a **geom function**.
3. A set of **aesthetics mappings** between variables in the data and visual properties in the geom function.

Plots are built by adding (+) layers one at a time.

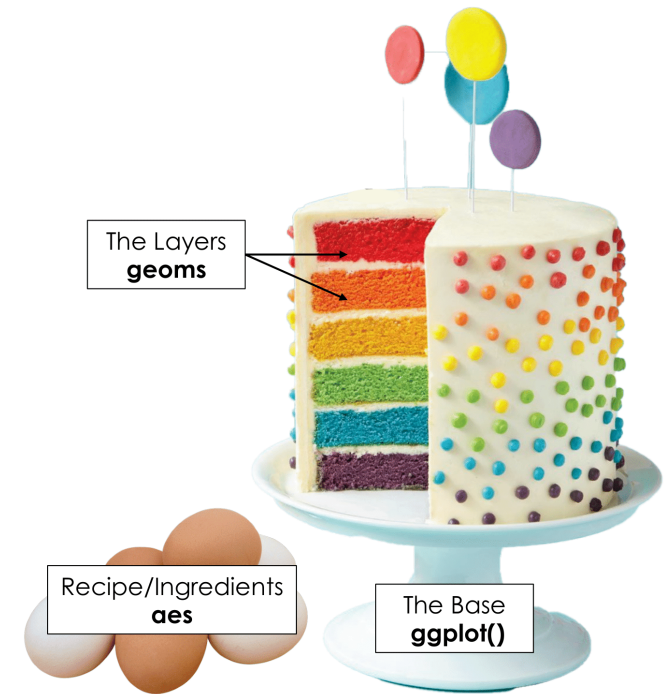
plotnine: Generating a Plot like a Cake

An **aesthetic** (**aes**) describes how data connect to visual properties of the plot:

- Position of a point.
- Size, shape, or colour of the points.
- Type, colour, or thickness of the lines.

A **geometrical object** (**geom**) defines how the data are presented visually:

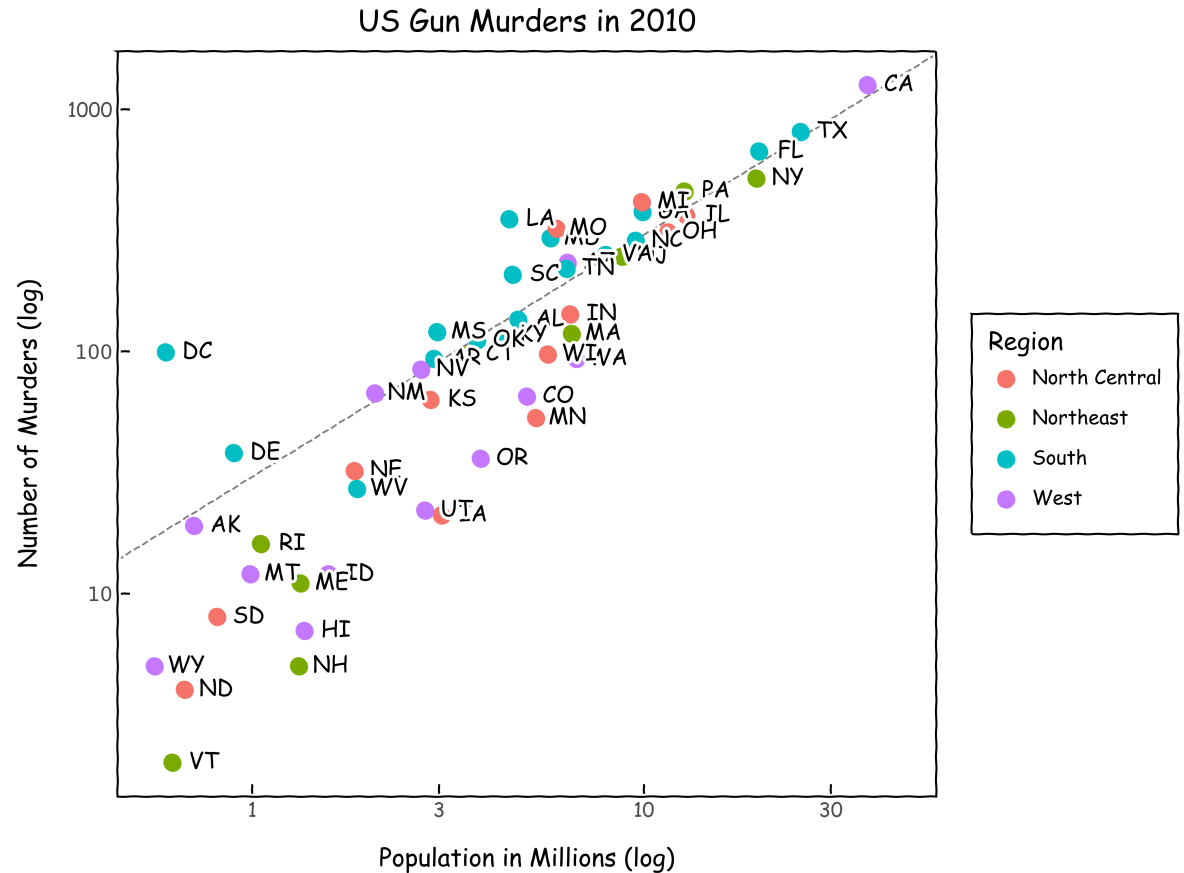
- Scatter plot uses the point geom.
- Bar chart uses the bar geom.



plotnine: Example of Visual Properties

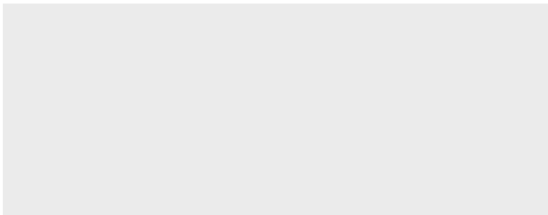
Here is what each visual property of the plot represents:

- Position of a point:
Population and total murders
- Colour of a point:
Region
- Text annotation:
Name of state
- Reference line:
Average murder rate

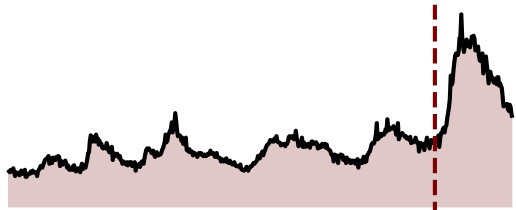


plotnine: How to Make a ~~Cake~~ Plot

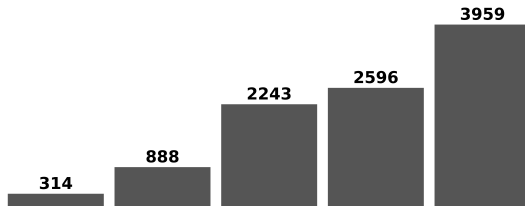
ggplot()



+ geom_line() + geom_area()
+ geom_vline()



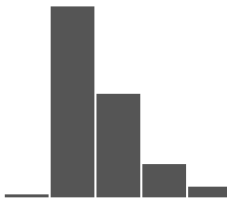
+ geom_bar() + geom_text()



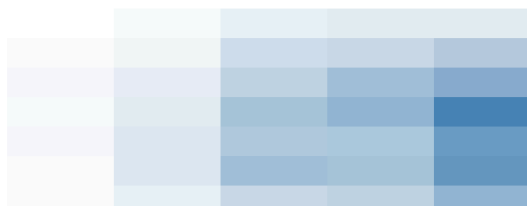
+ geom_point()



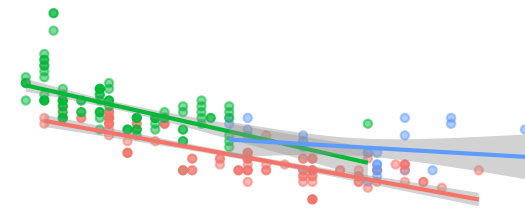
+ geom_histogram()



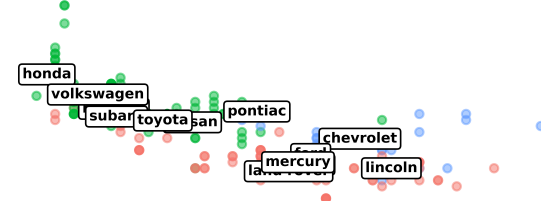
+ geom_tile()



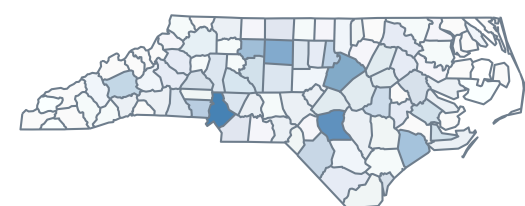
+ geom_point() + geom_smooth()



+ geom_point() + geom_label()



+ geom_map()



Relational Plots

Scatter Plot: `geom_point()`

`geom_point()` is used to create **scatter plots**.

The defining characteristic of a point is its position. So `x` and `y` are **required**. The other aesthetics are optional.

- `x` (required)
- `y` (required)
- `alpha`
- `color`
- `fill`
- `shape`
- `size`

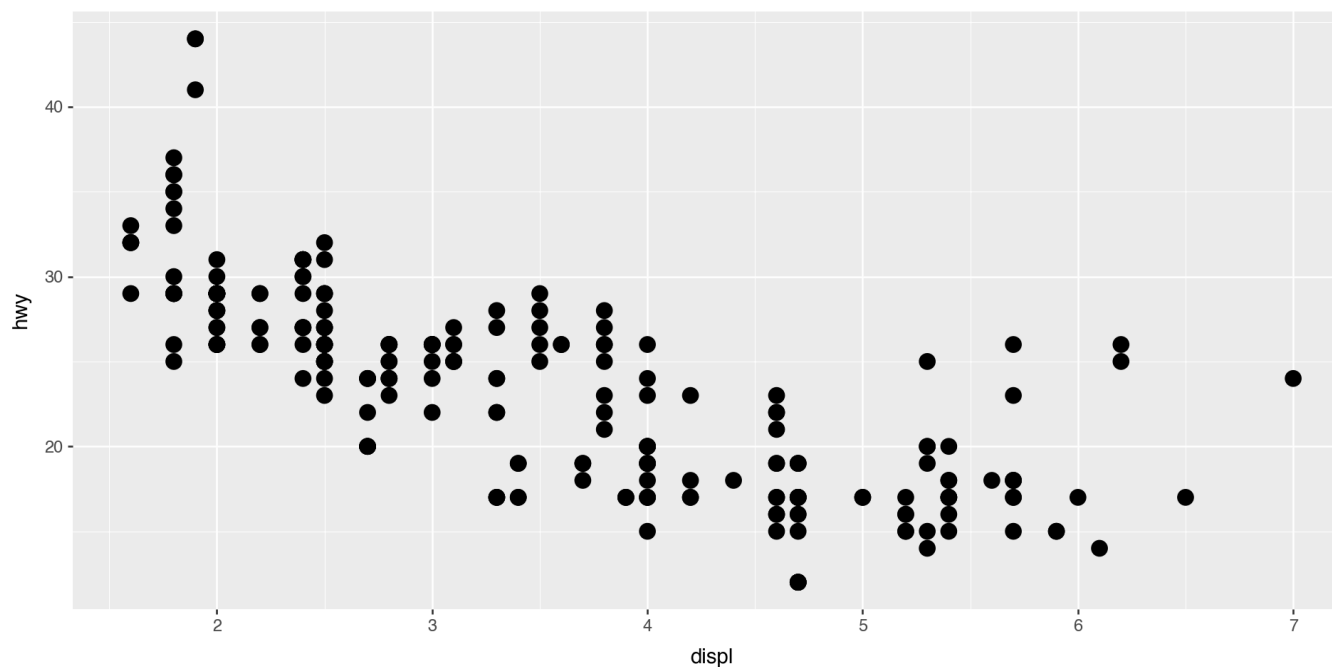
Scatter Plot: `geom_point()`

Basic Scatter Plot

A scatter plot between engine size and highway fuel efficiency.

- Why are `displ` and `hwy` mapped inside `aes()`, while `size` is outside?

```
1 ggplot(mpg) + geom_point(aes(x = 'displ', y = 'hwy'), size = 4)
```

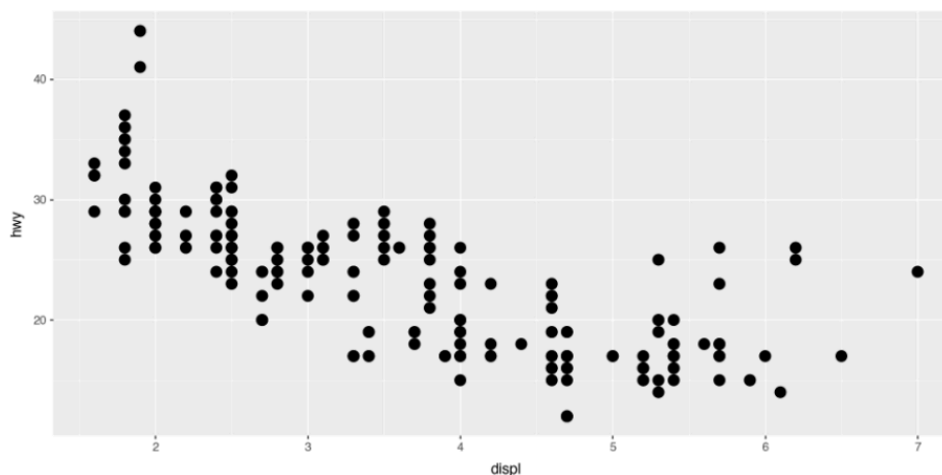


Scatter Plot: `geom_point()`

Basic Scatter Plot

```
ggplot(mpg) + geom_point(aes(x = 'displ', y = 'hwy'), size = 4)
```

	displ	hwy
0	1.8	29
1	1.8	29
2	2.0	31
3	2.0	30
4	2.8	26



- `aes()` is for **data mappings** — properties that vary based on variables in data.
- `size` is placed outside because it is a constant applied to all points.

This distinction is an important feature for `plotnine` visualisations.

Scatter Plot: `geom_point()`

Local vs Global Aesthetics Mapping

Compare the syntax below:

```
1 # Syntax 1: Local mapping
2 ggplot(mpg) + geom_point(aes(x = 'displ', y = 'hwy'), size = 4)
3
4 # Syntax 2: Global mapping
5 ggplot(mpg, aes(x = 'displ', y = 'hwy')) + geom_point(size = 4)
```

Pay attention to the structure of **Syntax 2**:

- Data and aesthetics mapping are both defined globally in `ggplot()`.
- Layer(s) added with `+` automatically inherits these mappings.

Scatter Plot: `geom_point()`

Global Aesthetics Mapping

Syntax 2 is the recommended practice.

```
1 # Syntax 2: Global mapping
2 ggplot(mpg, aes(x = 'displ', y = 'hwy')) + geom_point(size = 4)
```

As we build more complex plots with multiple layers, most layers will use the same `x` and `y` mappings.

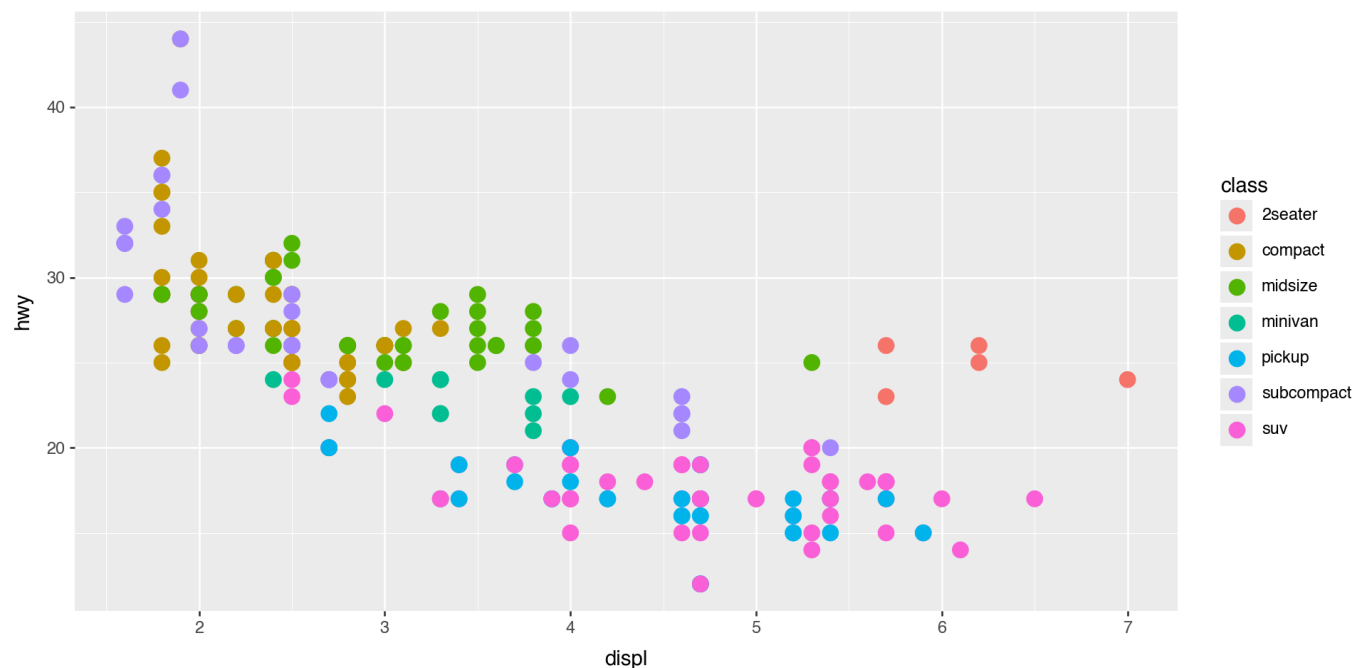
- Repeating them in every geom would be tedious.
- So we use **global aesthetic mapping**:
 - Define the aesthetics once in `ggplot()`.
 - All geoms will automatically default to these mappings.

Scatter Plot: `geom_point()`

Mapping Colour

- To visualise an additional variable, we can map it to the **color aesthetic**.
- A legend is created to show which colour corresponds to which value.

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) +  
2   geom_point(aes(color = 'class'), size = 4))
```

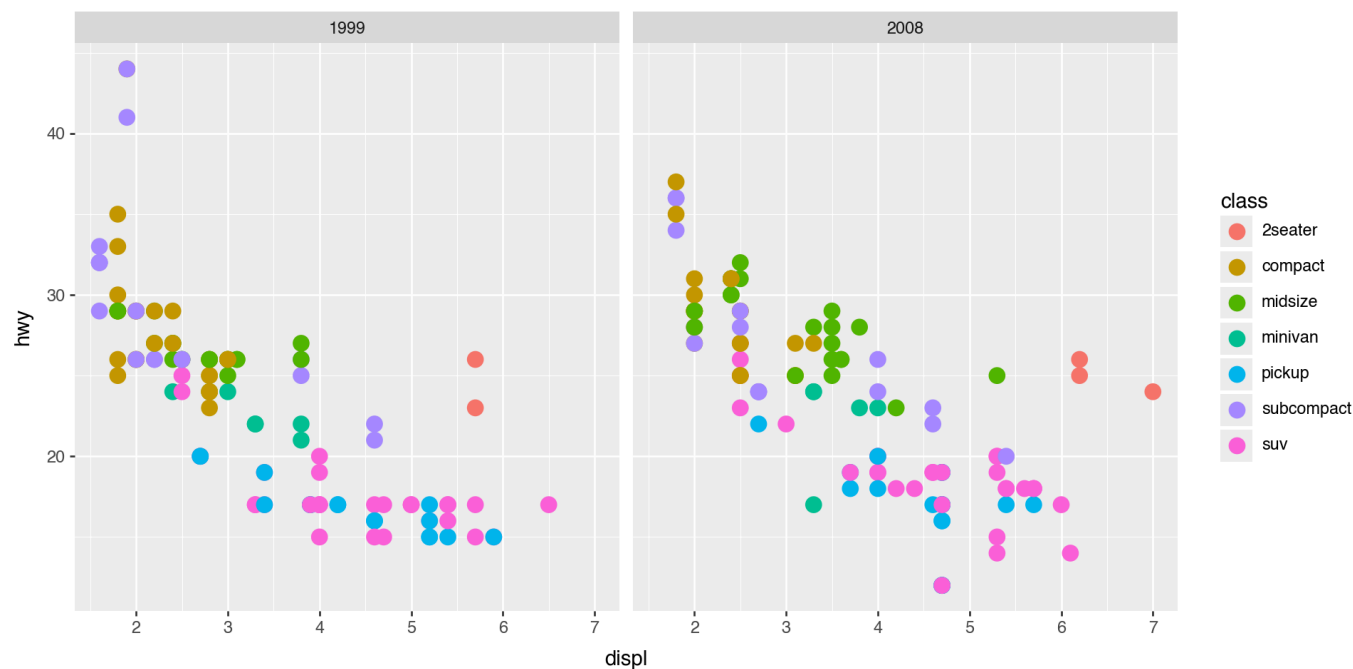


Scatter Plot: `geom_point()`

Facets

- We can further split the plot into **facets** with `facet_wrap()`.
- Each panel now represents a different *year*.

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) +  
2   geom_point(aes(color = 'class'), size = 4) + facet_wrap('year', nrow = 1))
```



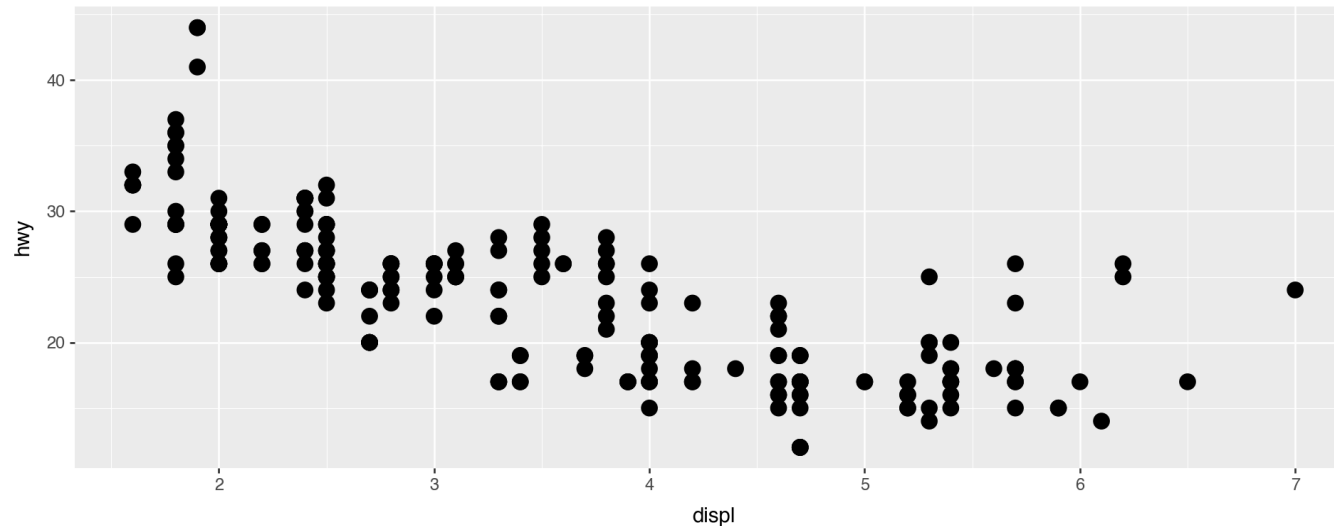
Scatter Plot: `geom_point()`

Handling Overplotting

`hwy` and `displ` are rounded so many points share the same coordinates.

- When many points overlap on each other, **overplotting** happens.
- Can be difficult to see how many observations fall in the same plot.

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) + theme(figure_size=(10, 4)) +  
2   geom_point(size = 4))
```

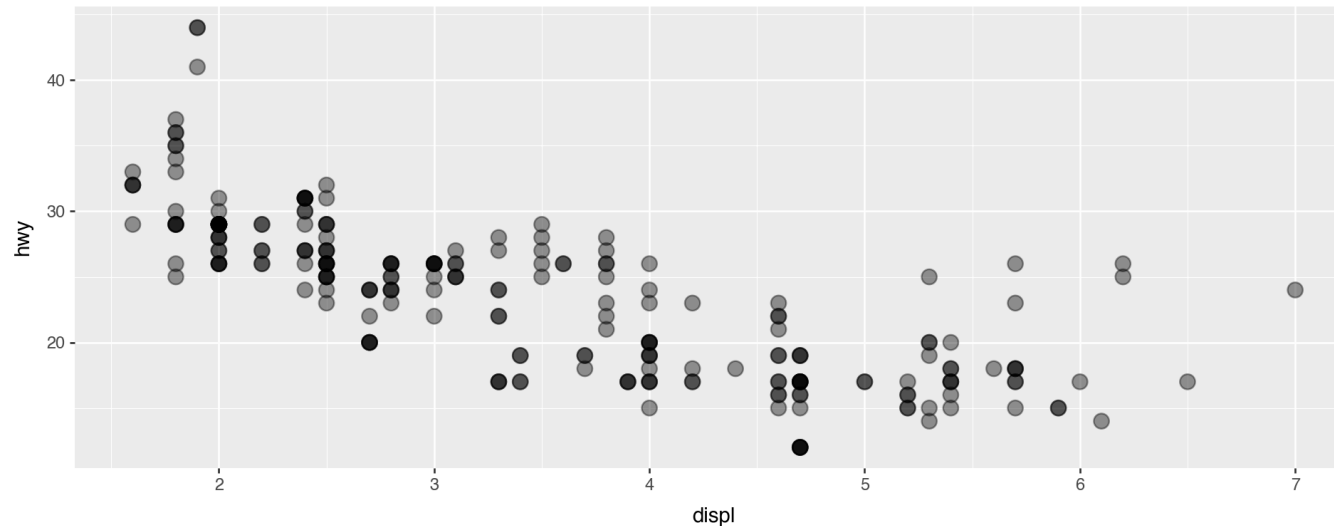


Scatter Plot: `geom_point()`

Handling Overplotting: Transparency

1. Use colour intensity (transparency) to show the density of the data.
 - Area appears darker when more points overlap with each other.
 - ... helps us see where data are more concentrated.

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) + theme(figure_size=(10, 4)) +  
2   geom_point(size = 4, alpha = 0.4))
```



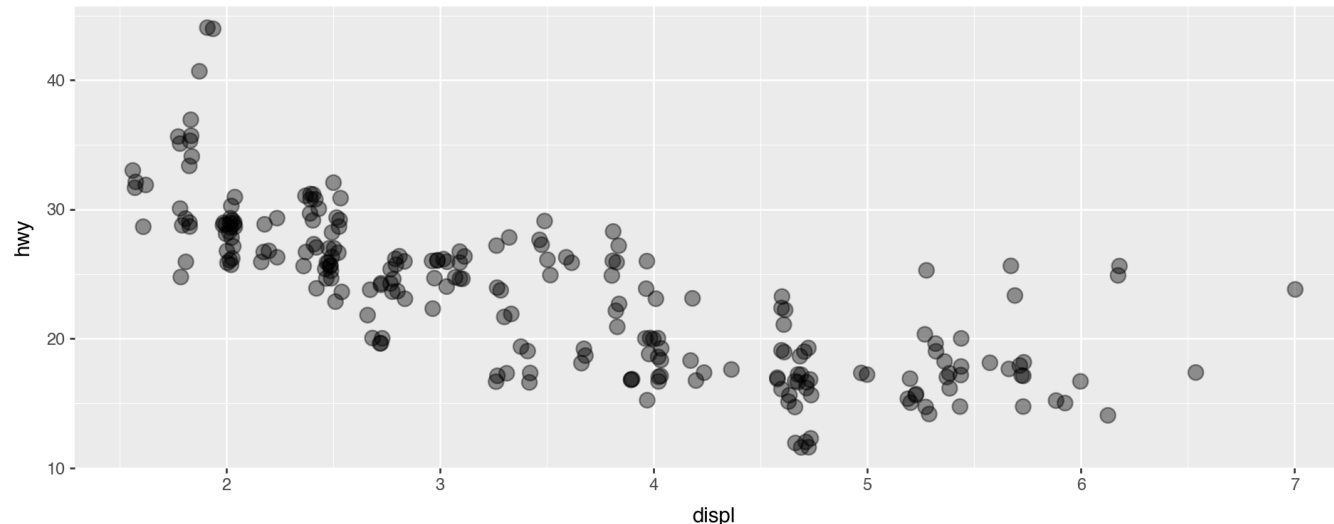
Scatter Plot: `geom_point()`

Handling Overplotting: Jitter

2. Set the position adjustment to `jitter`.

- This adds a small random noise to each point.
- ... spreads out the point so they are easier to see.

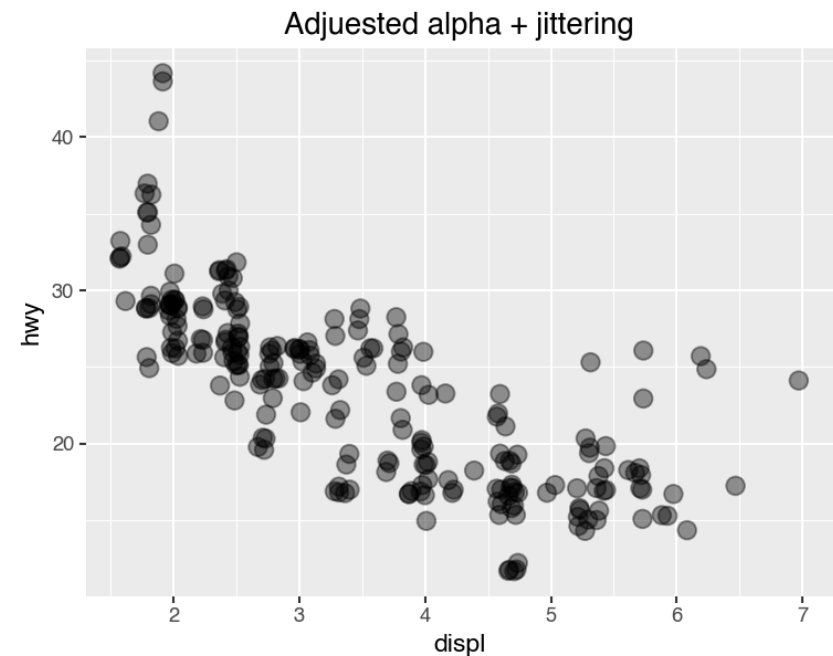
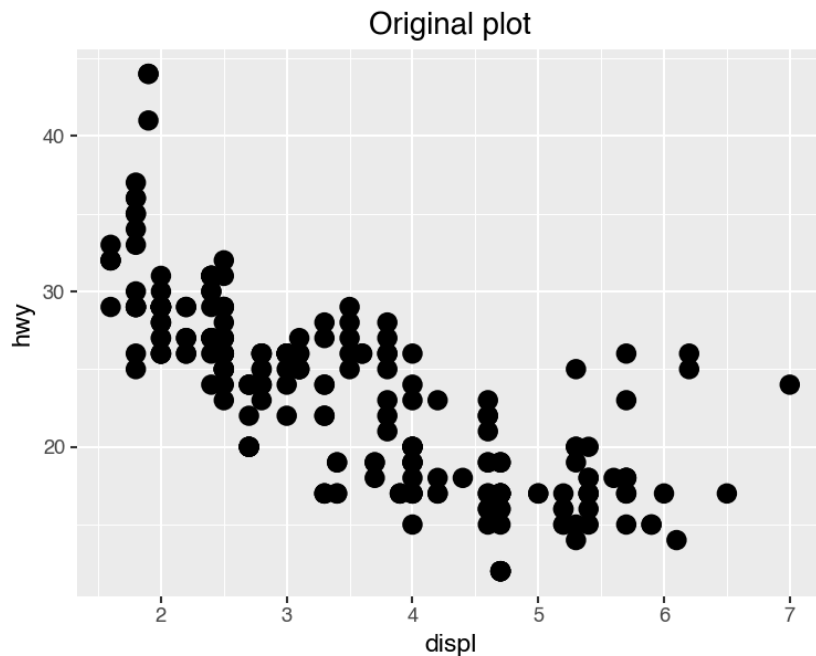
```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) + theme(figure_size=(10, 4)) +  
2   geom_point(size = 4, alpha = 0.4, position = 'jitter'))
```



Scatter Plot: `geom_point()`

Handling Overplotting: Comparison

- Transparency shows data density, and jittering further separates the overlapping points.
- In practice, we often combine both to make the pattern clearer without distorting the data.
- Jittering is just a visual adjustment and does not change the underlying data.

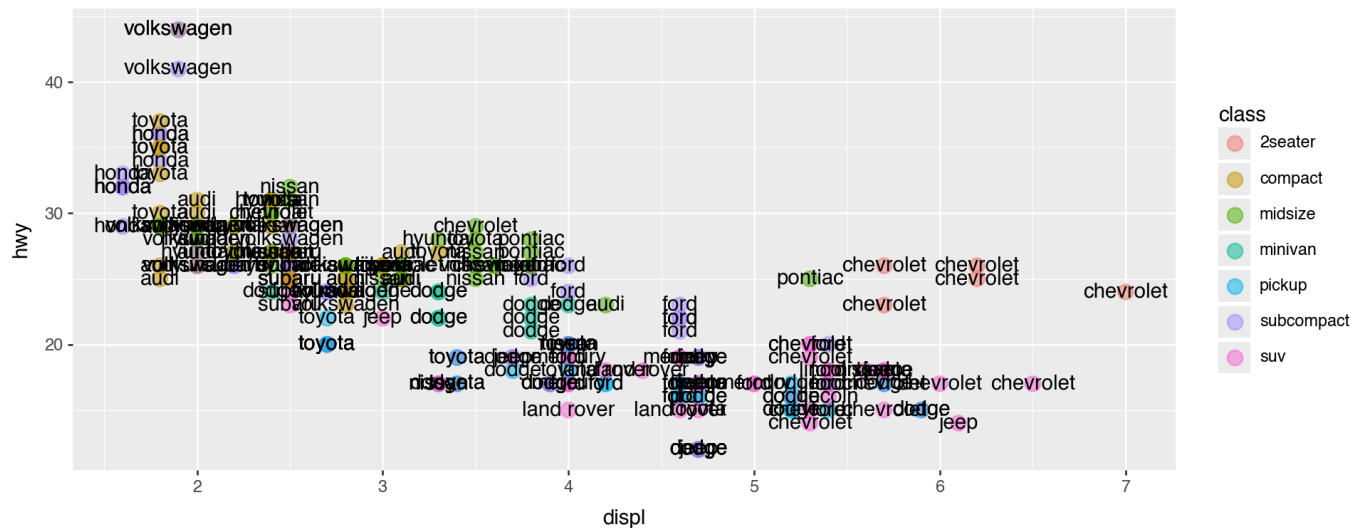


Scatter Plot: geom_point()

Annotations with geom_text() or geom_label()

- Adding labels to points can help highlight specific observations or groups.
- However, labeling every point can lead to visual cluster. 🚗

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) + theme(figsize=(10, 4)) +  
2   geom_point(aes(color = 'class'), size = 4, alpha = 0.5) +  
3   geom_text(aes(label = 'manufacturer')))
```



Scatter Plot: `geom_point()`

Annotations with `geom_text()` or `geom_label()`

- To resolve that, we will label **one representative point** per manufacturer.
- Let's prepare data for use with `geom_text()` or `geom_label()`.

```
1 df_mean = mpg.groupby('manufacturer')[['hwy', 'displ']].mean()  
2 df_mean = df_mean.reset_index()  
3 df_mean.head()
```

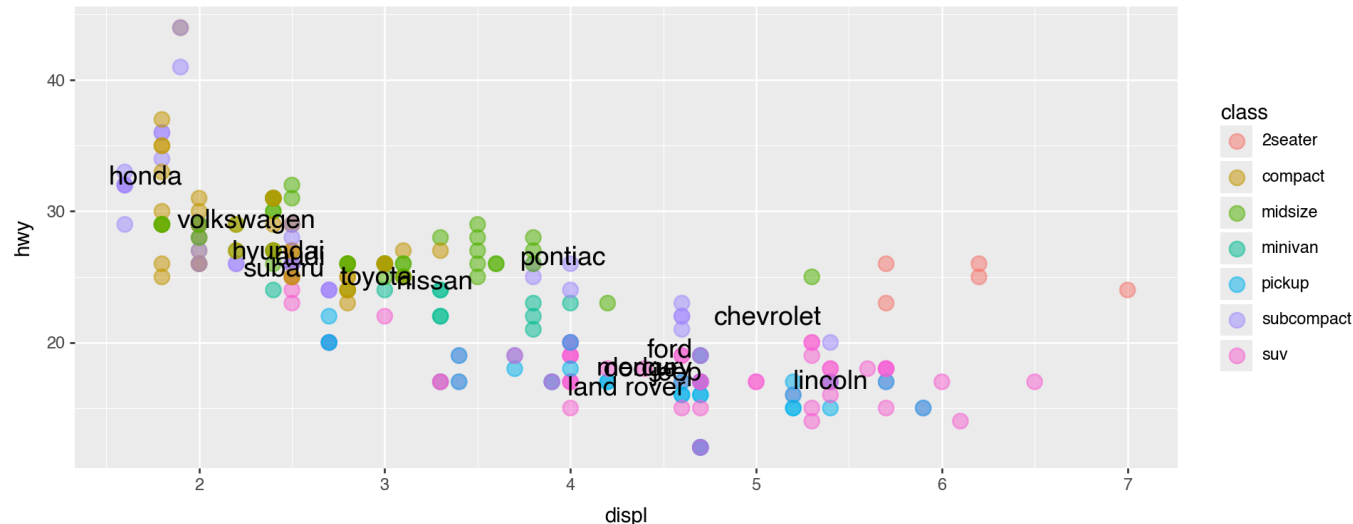
	manufacturer	hwy	displ
0	audi	26.444444	2.544444
1	chevrolet	21.894737	5.063158
2	dodge	17.945946	4.378378
3	ford	19.360000	4.536000
4	honda	32.555556	1.711111

Scatter Plot: `geom_point()`

Annotations with `geom_text()` or `geom_label()`

- We calculate the average engine size and fuel efficiency by manufacturer.
- Each manufacturer is now labeled only once on the plot.

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) + theme(figure_size=(10, 4)) +  
2   geom_point(aes(color = 'class'), size = 4, alpha = 0.5) +  
3   geom_text(aes(label = 'manufacturer'), data = df_mean,  
4             fontweight = 'bold', size = 14))
```

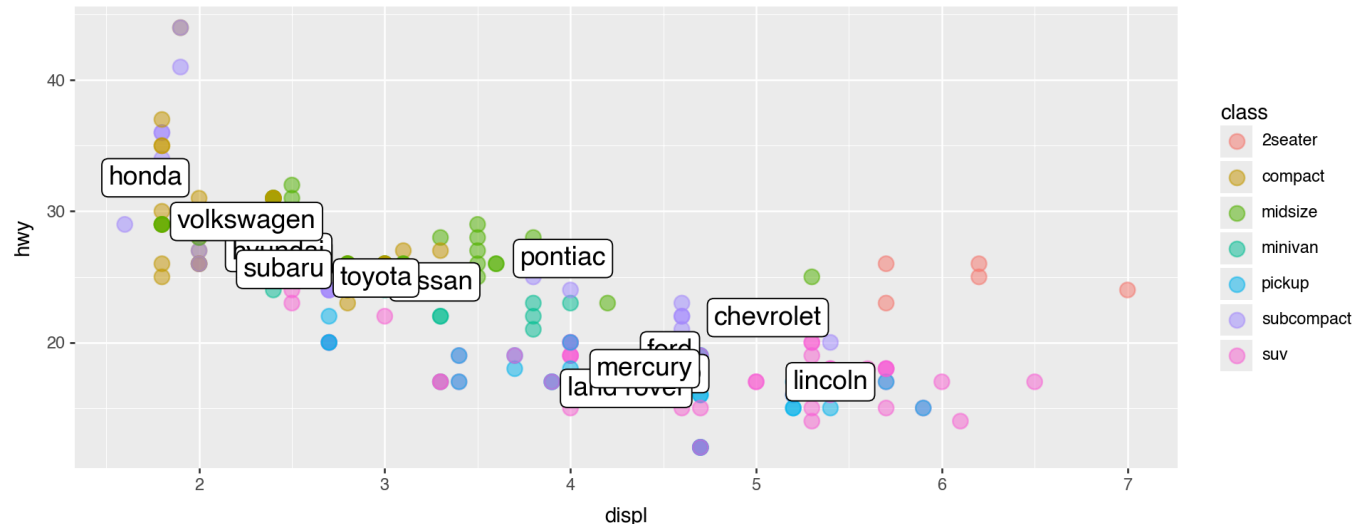


Scatter Plot: `geom_point()`

Annotations with `geom_text()` or `geom_label()`

- We can refine the plot with `geom_label()`
- ... which draws a rectangle behind the text for better readability.

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy')) + theme(figure_size=(10, 4)) +  
2   geom_point(aes(color = 'class'), size = 4, alpha = 0.5) +  
3   geom_label(aes(label = 'manufacturer'), data = df_mean,  
4             fontweight = 'bold', size = 14))
```



Scatter Plot: `geom_point()`

Title and Axis Labels

We will next see how to polish the overall plot with a `labs()` layer:

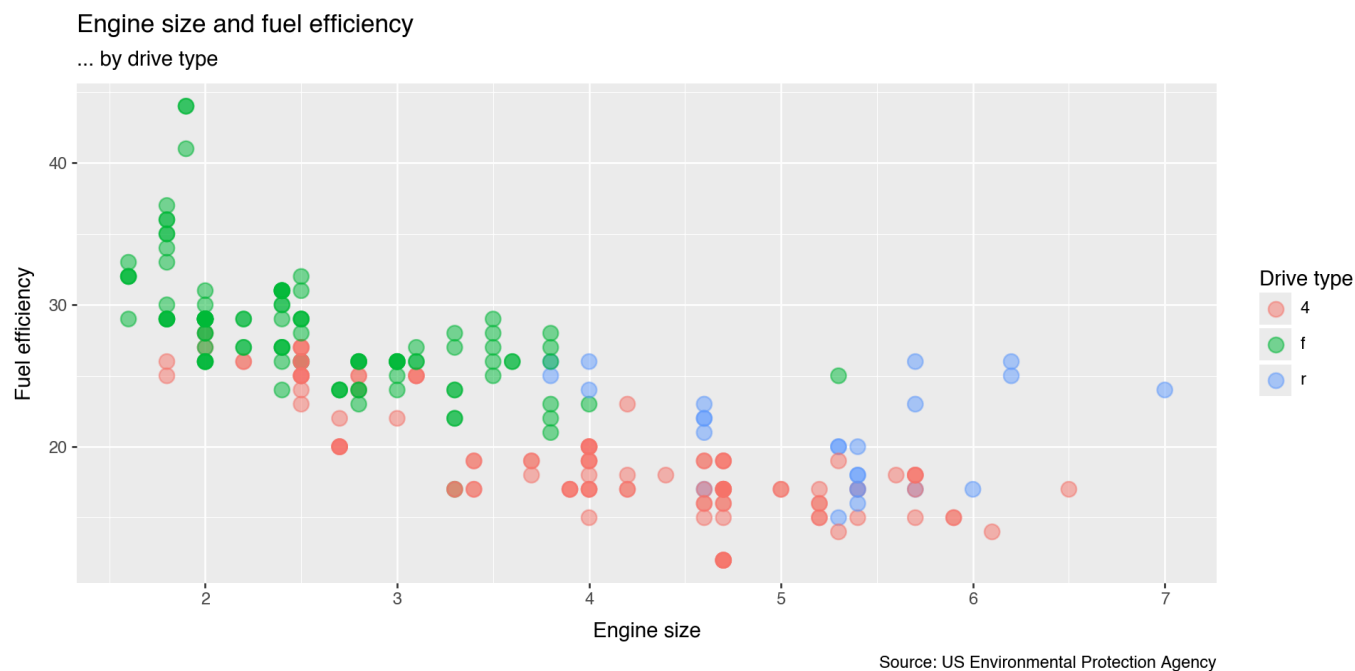
- `title`
- `subtitle`
- `caption`
- `x`
- `y`

We can also adjust legend titles for mapped aesthetics such as `color` or `fill`.

Scatter Plot: geom_point()

Title and Axis Labels

```
1 (ggplot(mpg, aes(x = 'displ', y = 'hwy', color = 'drv')) +  
2   geom_point(size = 4, alpha = 0.5) +  
3   labs(title = 'Engine size and fuel efficiency',  
4       subtitle = '... by drive type',  
5       caption='Source: US Environmental Protection Agency',  
6       x = 'Engine size', y = 'Fuel efficiency',  
7       color = 'Drive type'))
```



Saving a Plot as an Object

- We can save a plot as an object and add layers later with `plotnine`.
- The previous code is equivalent to the following:

```
1 # Saving the basic plot as an object
2 p = (ggplot(mpg, aes(x = 'displ', y = 'hwy', color = 'drv'))) +
3     geom_point(size = 4, alpha = 0.5))
```

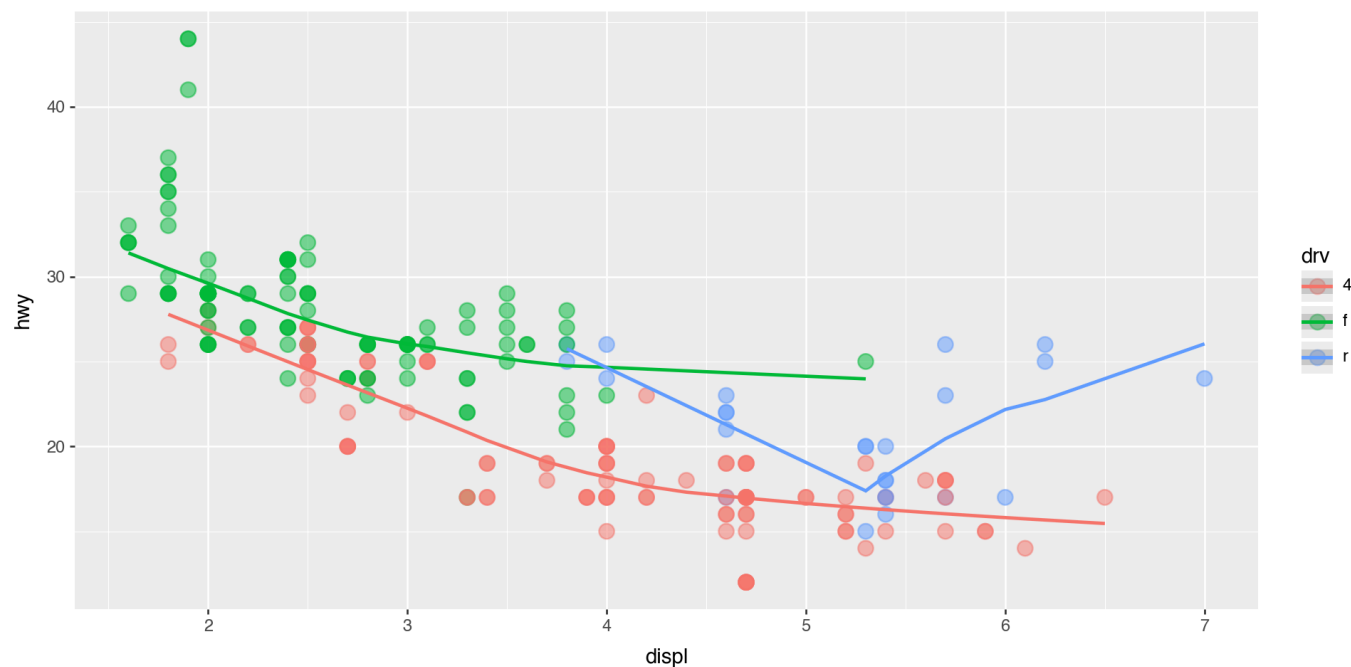
```
1 # Adding title and labels subsequently
2 (p + labs(title = 'Engine size and fuel efficiency',
3           subtitle = '... by drive type',
4           caption='Source: US Environmental Protection Agency',
5           x = 'Engine size', y = 'Fuel efficiency',
6           color = 'Drive type'))
```

Scatter Plot: `geom_point()`

Trend Line with `geom_smooth()`: Smooth Trend

- `... + geom_smooth()` adds a fitted trend line to the scatter plot `p`.
- The `geom_smooth()` layer adopts the aesthetic mappings defined earlier.

```
1 p + geom_smooth()
```

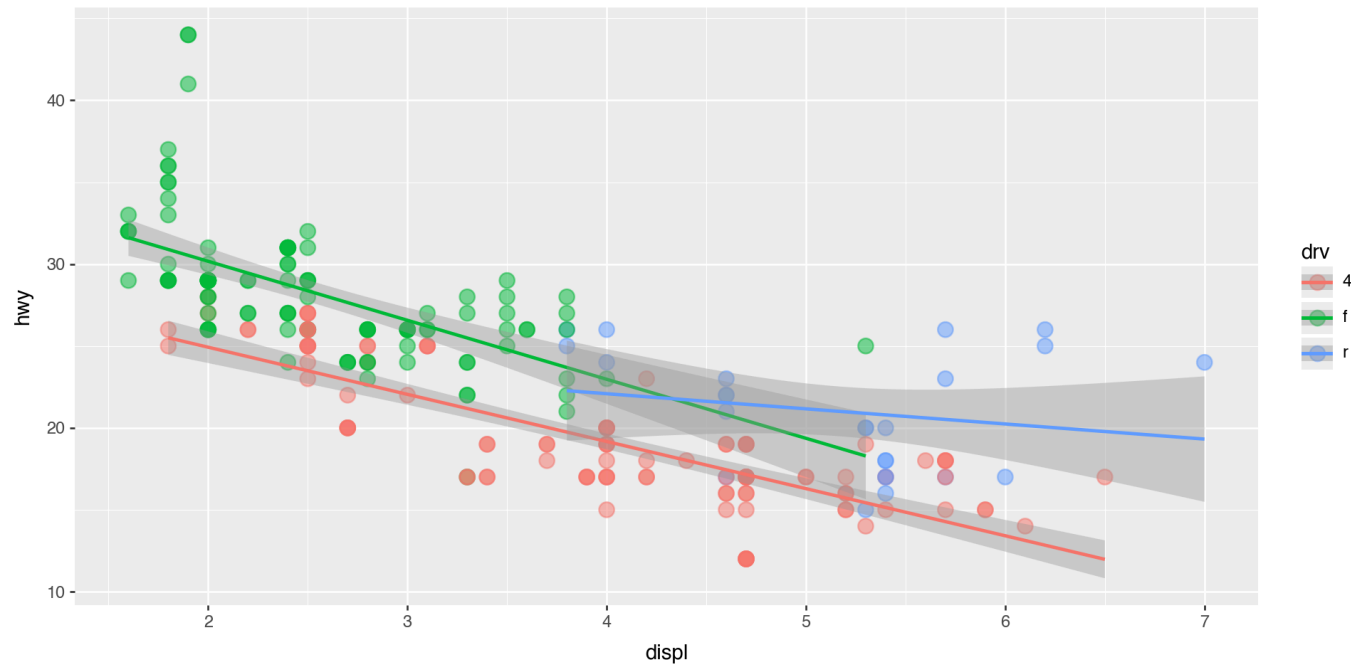


Scatter Plot: `geom_point()`

Trend Line with `geom_smooth()`: Linear Model

- We can fit a straight line by specifying a linear model (`method = 'lm'`).
- It also displays a confidence band automatically.

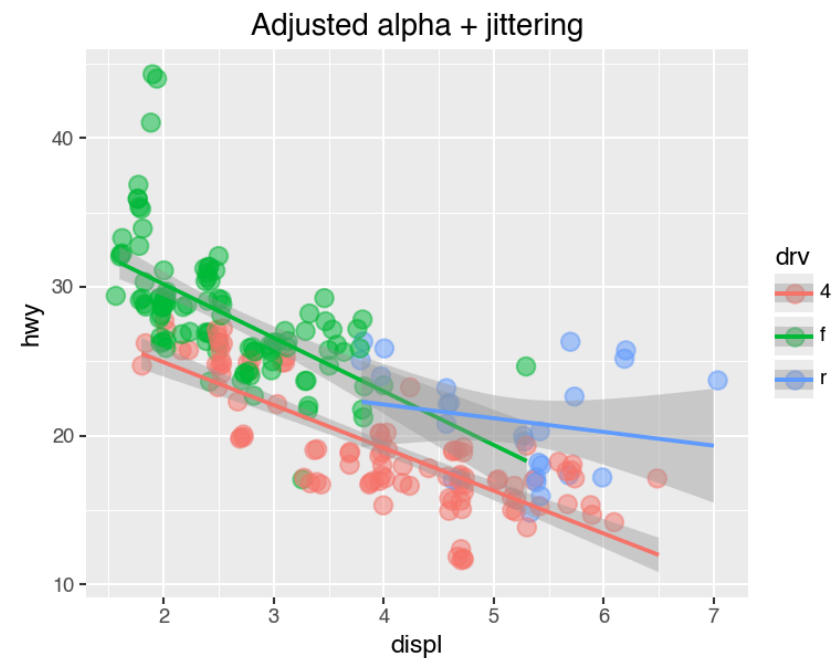
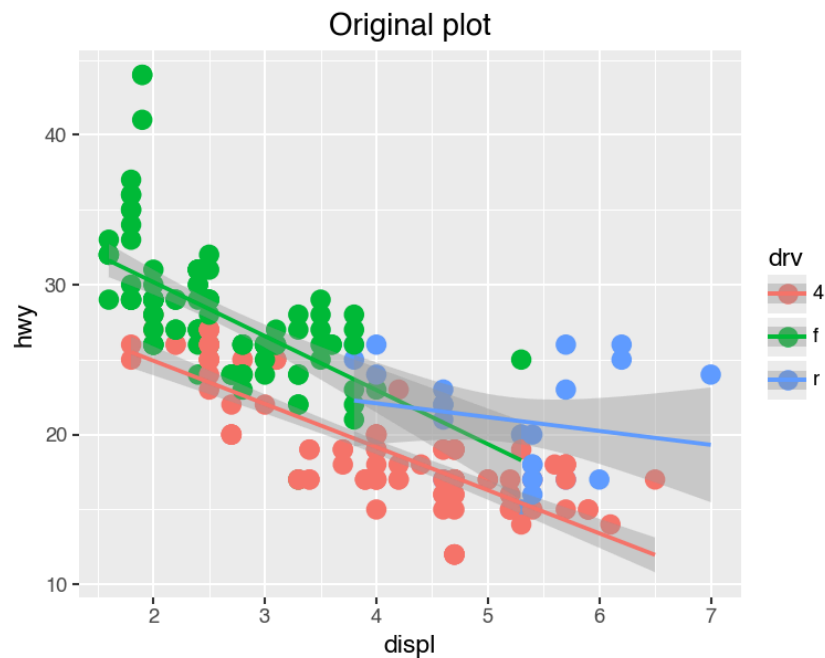
```
1 p + geom_smooth(method = 'lm')
```



Scatter Plot: `geom_point()`

Trend Line with `geom_smooth()`: Jittering and Trends

- Jittering only applies a visual adjustment and does not change the underlying data or the fitted trend.
- The trend lines are still based on the original data values.



Line Plot: `geom_line()`

`geom_line()` connects observations in the order of the `x` variable, usually a date or time.

It is ideal for time-series data. The aesthetics that the line geom uses are

- `x` (required)
- `y` (required)
- `alpha`
- `color`
- `size`

Line Plot: `geom_line()`

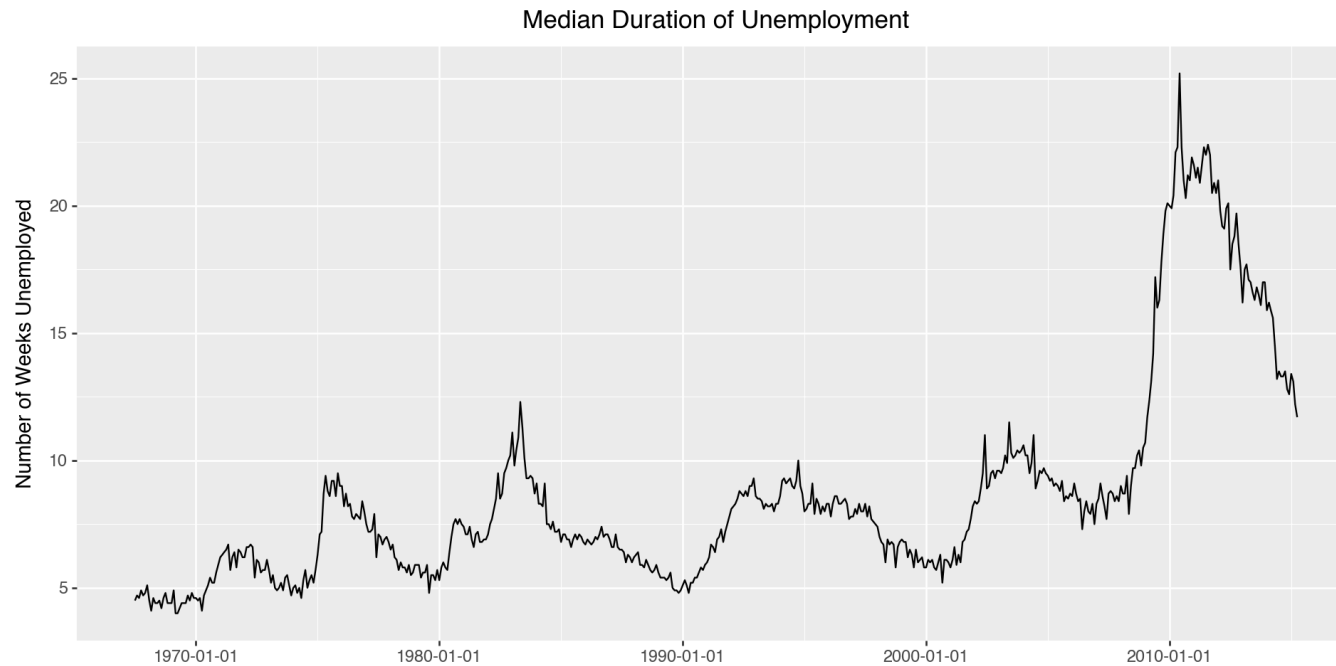
- Let us use the `economics` data from `plotnine`.
- US economic time series data from 1967 to 2015.

```
1 economics = pd.read_csv('../data/economics.csv', parse_dates=[0])  
2 economics.head()
```

	date	pce	pop	psavert	uempmed	unemploy
0	1967-07-01	507.4	198712	12.5	4.5	2944
1	1967-08-01	510.5	198911	12.5	4.7	2945
2	1967-09-01	516.3	199113	11.7	4.6	2958
3	1967-10-01	512.9	199311	12.5	4.9	3143
4	1967-11-01	518.1	199498	12.5	4.7	3066

Line Plot: geom_line()

```
1 (ggplot(economics, aes(x = 'date', y = 'uempmed')) +  
2   geom_line() +  
3   labs(title = 'Median Duration of Unemployment',  
4         x = '', y = 'Number of Weeks Unemployed'))
```



Line Plot: `geom_line()`

Customisation

```
1 (ggplot(economics, aes(x = 'date', y = 'uempmed')) +  
2   geom_line(size = 1) +  
3   labs(title = 'Median Duration of Unemployment',  
4         x = '', y = 'Number of Weeks Unemployed') +  
5   scale_x_datetime(date_breaks = '10 years',  
6                     date_labels = '%Y'))
```

- In the `geom_line()` layer, `size = 1` controls the thickness of the line.
- `scale_x_datetime()` modifies the appearance of the x-axis.
 - `date_breaks = '10 years'` adds ticks for every 10 years.
 - `date_labels = '%Y'` displays only the year information on the axis.

Line Plot: `geom_line()`

Customisation



Line Plot: `geom_line()`

Reference Lines, Annotations

To highlight key events, we can add reference lines and text annotations.

- `geom_vline()` for vertical lines.
- `geom_hline()` for horizontal lines.
- `geom_abline()` for straight lines defined by a slope or an intercept.
 - The `ab` in the name refers to the intercept (`a`) and the slope (`b`).

We can use an `annotate()` layer to add texts at specific coordinates.

Line Plot: `geom_line()`

Reference Lines, Annotations

```
1 (ggplot(economics, aes(x = 'date', y = 'uempmed')) +  
2   geom_line(size = 1) +  
3   geom_vline(xintercept = '2007-12-01',  
4             linetype = 'dashed', color = 'maroon', size = 1) +  
5   annotate('text', x = '2007-12-01', y = 20,  
6           label = 'Global Financial Crisis',  
7           angle = 90, ha = 'right', color = 'maroon') +  
8   labs(title = 'Median Duration of Unemployment',  
9        x = '', y = 'Number of Weeks Unemployed') +  
10  scale_x_datetime(date_breaks = '10 years',  
11                  date_labels = '%Y'))
```

- `xintercept` in `geom_vline()` specifies vertical line/s to be drawn.
- `annotate()` adds text to the plot.
 - `angle = 90` rotates text by 90 degrees.
 - `ha = 'right'` aligns the text horizontally to the right.

Line Plot: `geom_line()`

Reference Lines, Annotations



Line Plot: `geom_line()`

Area under Line

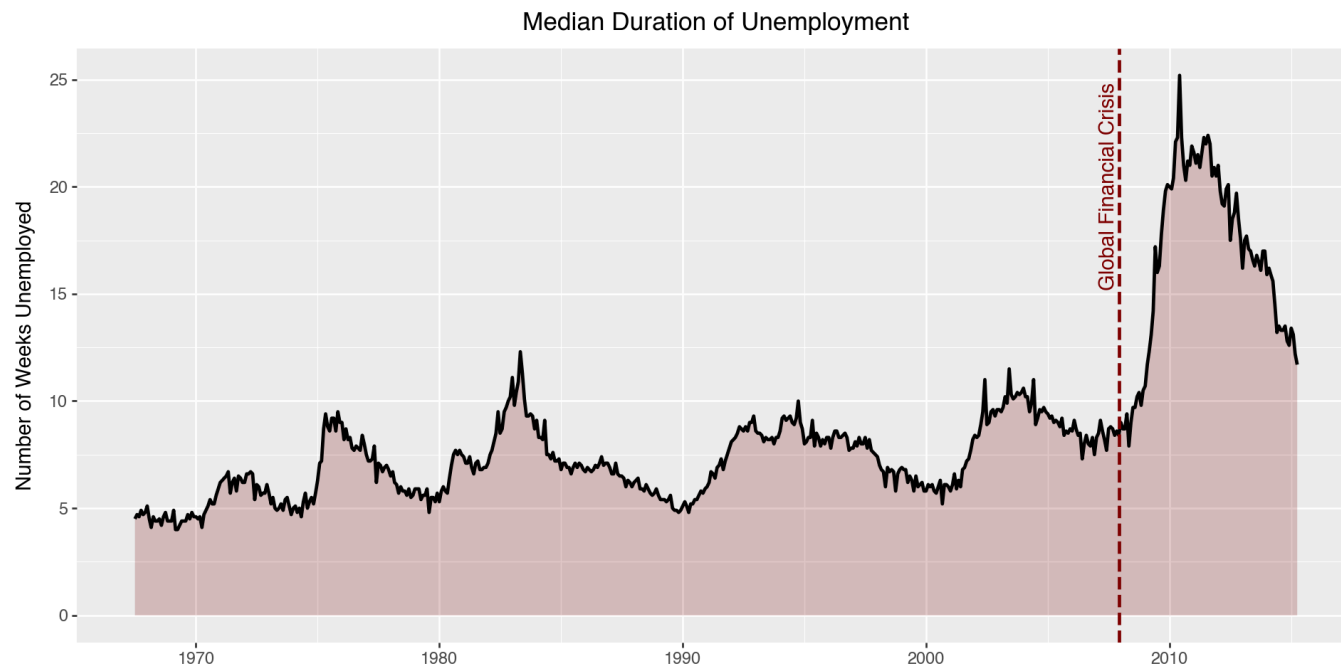
- Shading area under line emphasises magnitude and changes over time.
- A layer of `geom_area()` is added to the plot.

```
1 (ggplot(economics, aes(x = 'date', y = 'uempmed')) +  
2   geom_area(fill = 'maroon', alpha = 0.2) +  
3   geom_line(size = 1) +  
4   geom_vline(xintercept = '2007-12-01',  
5             linetype = 'dashed', color = 'maroon', size = 1) +  
6   annotate('text', x = '2007-12-01', y = 20,  
7           label = 'Global Financial Crisis',  
8           angle = 90, ha = 'right', color = 'maroon') +  
9   labs(title = 'Median Duration of Unemployment',  
10        x = '', y = 'Number of Weeks Unemployed') +  
11  scale_x_datetime(date_breaks = '10 years', date_labels = '%Y'))
```



Line Plot: `geom_line()`

Area under Line



Continuous Distributions

Histogram: `geom_histogram()`

`geom_histogram()` is used to create histograms. It allows us to visualise **continuous** variables.

The aesthetic arguments are

- `x` (required)
- `alpha`
- `color`
- `fill`

Histogram: `geom_histogram()`

Diamonds Data

The data set `diamonds` records prices and physical attributes of 53000+ diamonds.

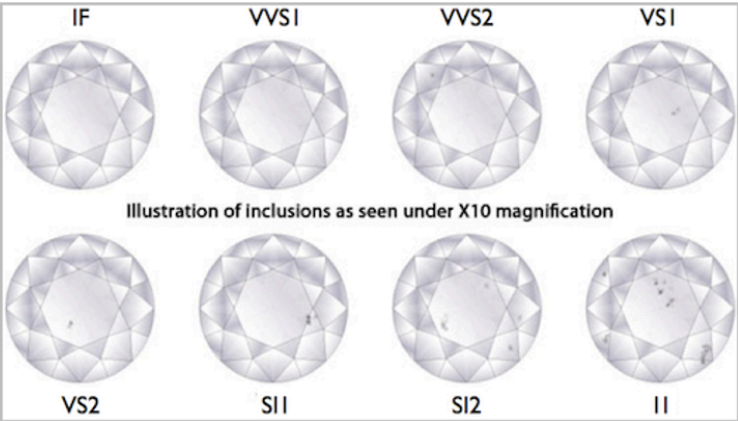
We will however only work with a sample of 10000 observations today.

```
1 diamonds = pd.read_csv("../data/diamonds.csv")
2 diamonds = diamonds.sample(10000, random_state = 1029)
3 diamonds.head()
```

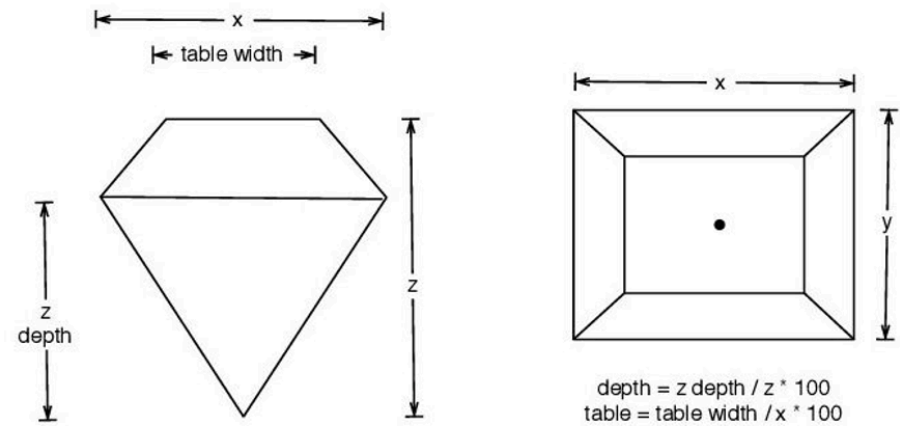
	carat	cut	color	clarity	depth	table	price	x	y	z
17317	1.15	Premium	D	SI1	63.0	57.0	6942	6.69	6.64	4.20
20541	1.15	Premium	G	VVS1	60.8	58.0	8863	6.74	6.77	4.11
48647	0.34	Ideal	H	VS2	61.7	57.0	537	4.46	4.48	2.76
24306	2.00	Good	H	SI2	63.8	59.0	12554	7.93	7.88	5.04
4440	1.00	Good	J	SI1	59.5	61.0	3616	6.46	6.52	3.86

Histogram: geom_histogram()

Diamonds Data: Physical Attributes



COLOR GRADING SCALE																									
																									
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z			
Colorless			Near Colorless				Faint Yellow			Very Light Yellow					Light Yellow										

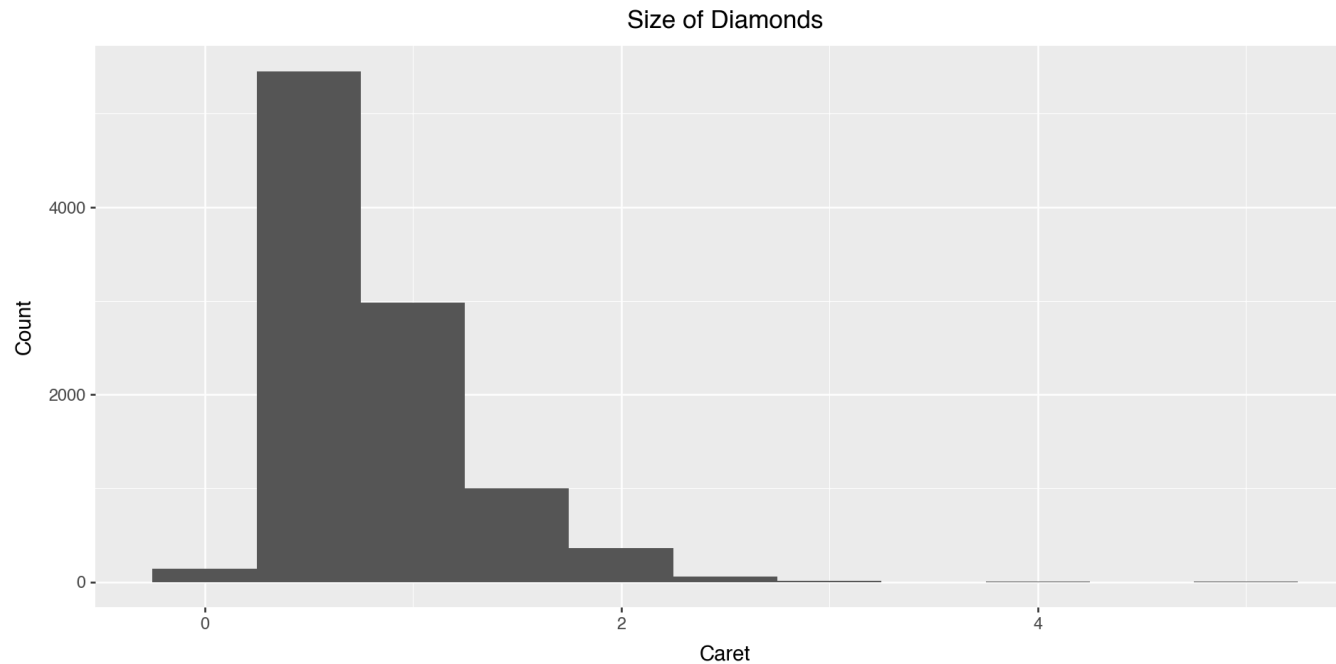


Histogram: geom_histogram()

Diamonds Data: Size

A large bin width means each bar covers a wider range of carat sizes.

```
1 (ggplot(diamonds, aes(x = 'carat')) +  
2   geom_histogram(binwidth = 0.5) +  
3   labs(title = 'Size of Diamonds', x = 'Carat', y = 'Count'))
```

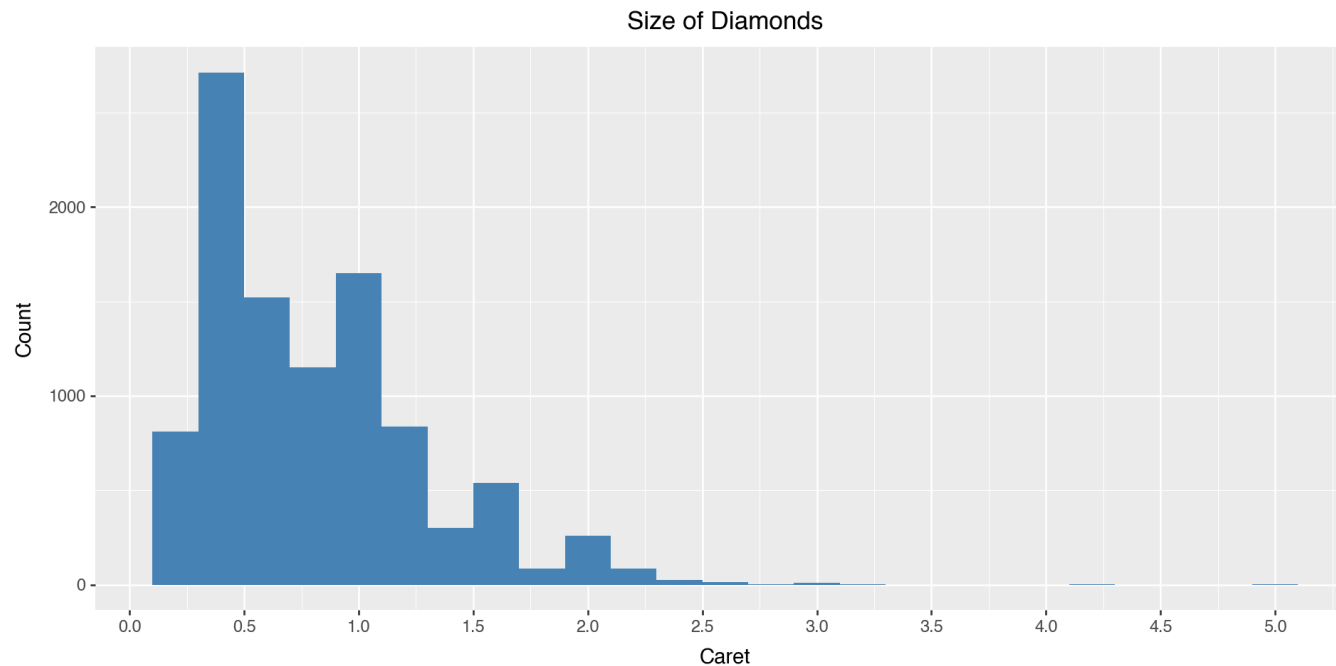


Histogram: geom_histogram()

Diamonds Data: Size

By reducing the bin width, we see the distribution in finer detail.

```
1 (ggplot(diamonds, aes(x = 'carat')) +  
2   geom_histogram(binwidth = 0.2, fill = 'steelblue') +  
3   labs(title = 'Size of Diamonds', x = 'Carat', y = 'Count') +  
4   scale_x_continuous(breaks = np.arange(0, 5.5, 0.5)))
```

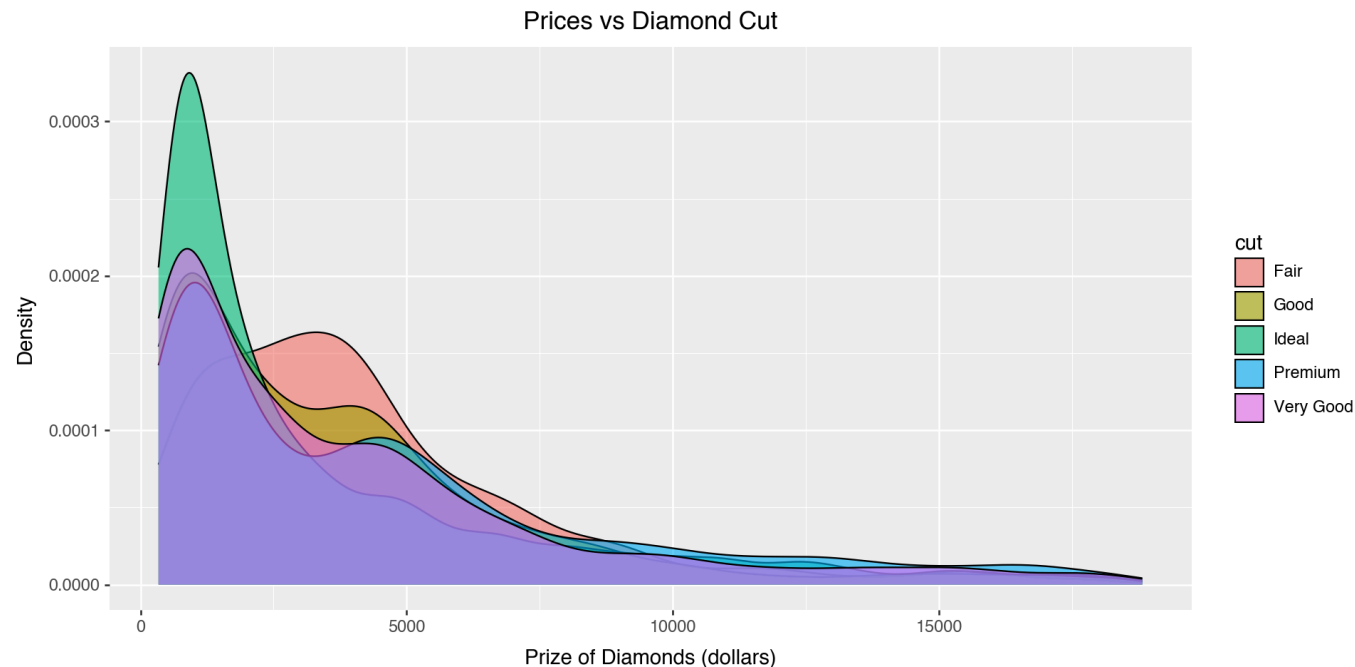


Kernel Density Estimation Plot: `geom_density()`

Diamonds Data: Compare Groups

With `geom_density()`, we can better compare the distributions by groups.

```
1 (ggplot(diamonds, aes(x = 'price', fill = 'cut')) +  
2   geom_density(alpha = 0.6) +  
3   labs(title = 'Prices vs Diamond Cut', x = 'Prize of Diamonds (dollars)',  
4     y = 'Density'))
```



Kernel Density Estimation Plot: `geom_density()`

Diamonds Data: Compare Groups using Facets

We use `facet_wrap()` to produce one panel for each category.

```
1 (ggplot(diamonds, aes(x = 'price', fill = 'cut')) +  
2   geom_density(alpha = 0.6, show_legend = False) +  
3   labs(title = 'Prices vs Diamond Cut', x = 'Prize of Diamonds (dollars)',  
4     y = 'Density') + facet_wrap('cut', ncol = 5))
```



Categorical Distributions

Bar Chart: `geom_bar()`

`geom_bar()` is used to create bar charts. It allows us to visualise **categorical** variables.

The aesthetic arguments are

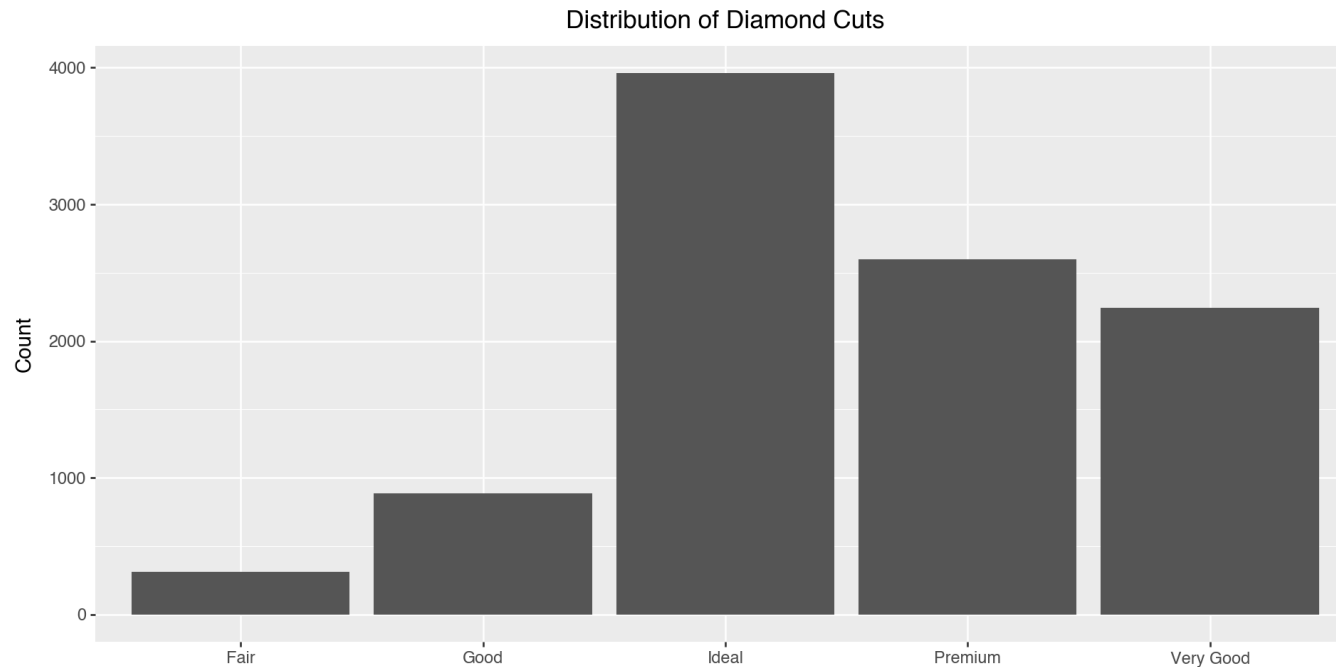
- `x` (required)
- `alpha`
- `fill`

Bar Chart: geom_bar()

Distribution of Cut

The `cut` variable is categorical – suitable for a bar chart.

```
1 (ggplot(diamonds, aes(x = 'cut')) + geom_bar() +  
2   labs(title = 'Distribution of Diamond Cuts', x = '', y = 'Count'))
```

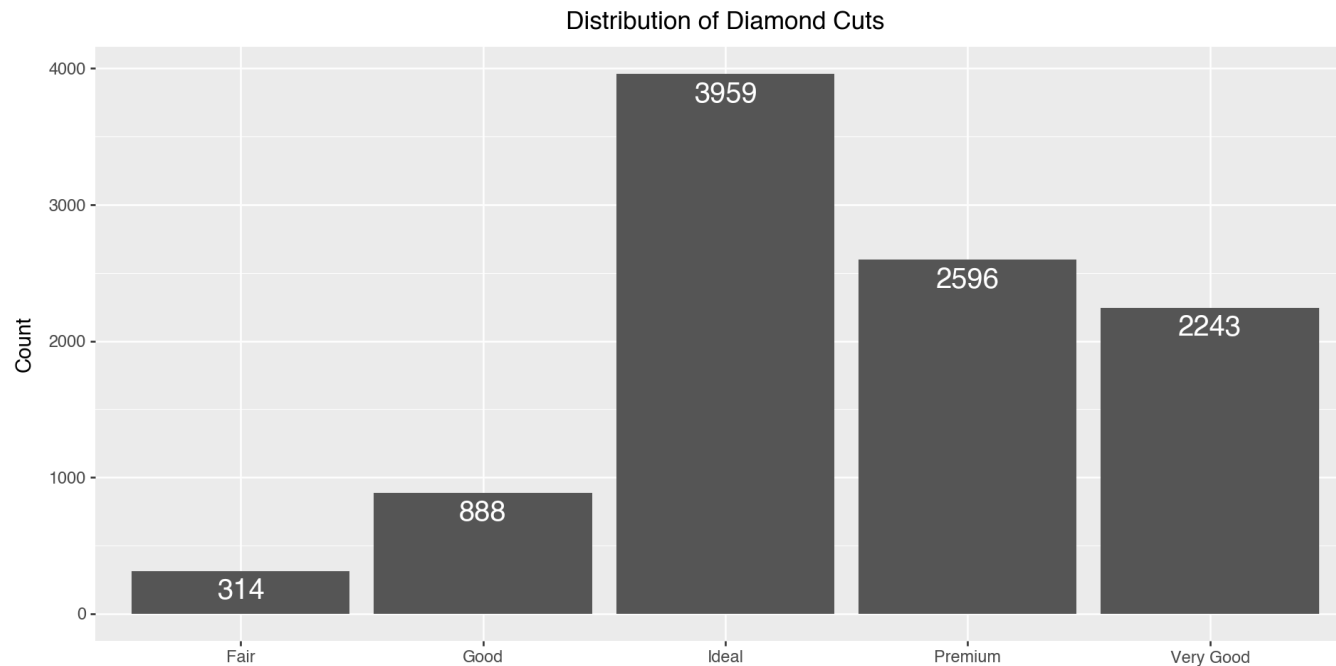


Bar Chart: geom_bar()

Distribution of Cut

We can make the bar chart more informative by adding value labels.

```
1 df_count = diamonds['cut'].value_counts().reset_index()
2 (ggplot(diamonds, aes(x = 'cut')) + geom_bar() +
3     labs(title = 'Distribution of Diamond Cuts', x = '', y = 'Count') +
4     geom_text(aes(y = 'count', label = 'count'), data = df_count,
5               nudge_y = -150, size = 15, color = 'white', fontweight = 'bold'))
```

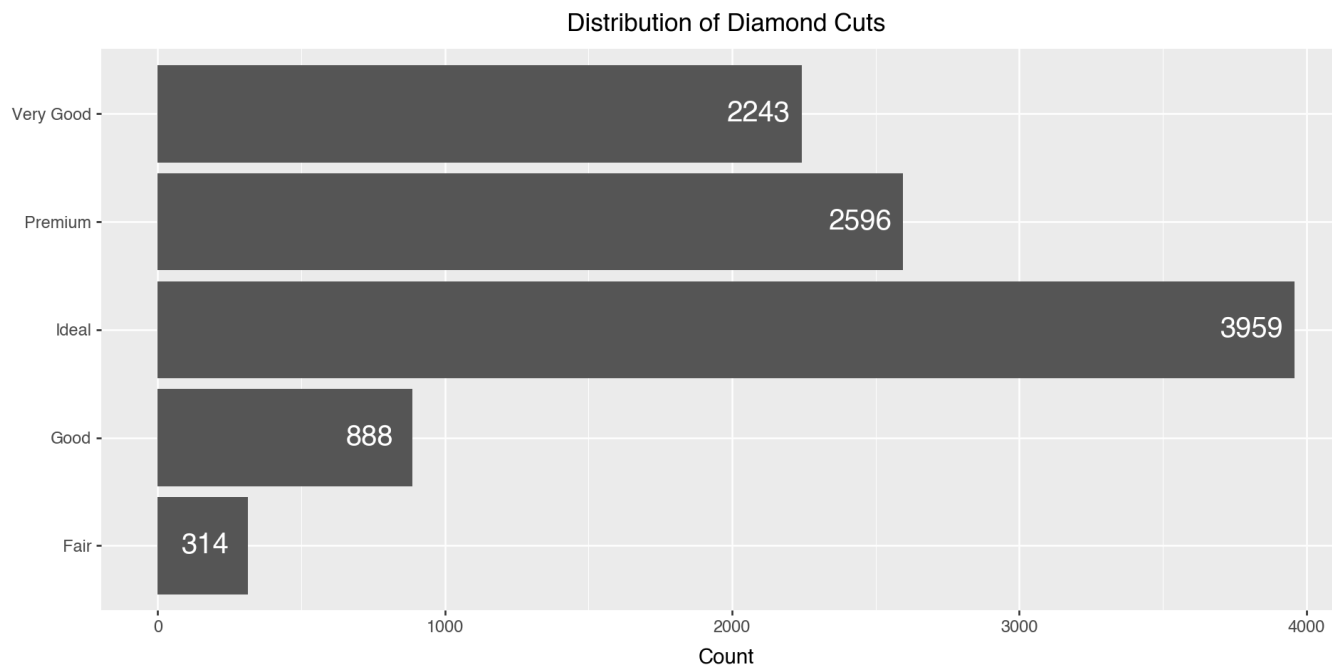


Bar Chart: `geom_bar()`

Distribution of Cut

Using `coord_flip()` to flip the coordinates make the chart easier to read.

```
1 (ggplot(diamonds, aes(x = 'cut')) + geom_bar() +  
2   labs(title = 'Distribution of Diamond Cuts', x = '', y = 'Count') +  
3   geom_text(aes(y = 'count', label = 'count'), data = df_count,  
4             nudge_y = -150, size = 15, color = 'white', fontweight = 'bold') +  
5   coord_flip())
```



Heatmap: `geom_tile()`

`geom_tile()` is used to create heatmaps.

Each tile represents a combination of categories, and colour shows the magnitude.

The aesthetic arguments are

- `x`
- `y`
- `fill`: Numeric value used to determine colour intensity.

Heatmap: geom_tile()

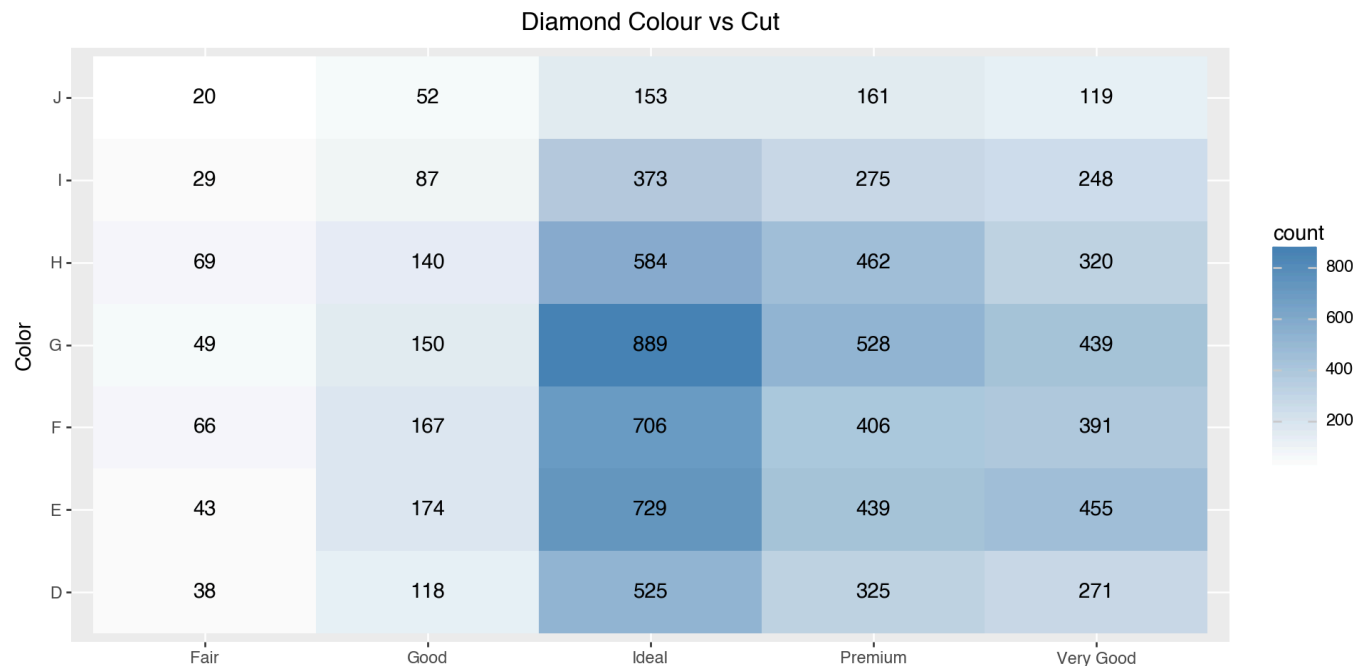
We first prepare the data for this geom.

```
1 df_count1 = (diamonds
2     .groupby(['cut', 'color'])
3     .size()
4     .reset_index(name = 'count'))
5 df_count1.head()
```

	cut	color	count
0	Fair	D	38
1	Fair	E	43
2	Fair	F	66
3	Fair	G	49
4	Fair	H	69

Heatmap: geom_tile()

```
1 (ggplot(df_count1, aes(x = 'cut', y = 'color', fill = 'count')) +  
2   geom_tile() + geom_text(aes(label = 'count')) +  
3   labs(x = '', y = 'Color', title = 'Diamond Colour vs Cut') +  
4   scale_fill_gradient(low = 'white', high = 'steelblue'))
```



Choropleth Maps

Choropleth Maps: `geom_map()`

Let us explore how to create a **choropleth map** with `geom_map()`.

- Works like a heatmap on a map.

To plot geographic data, we need a `GeoDataFrame`

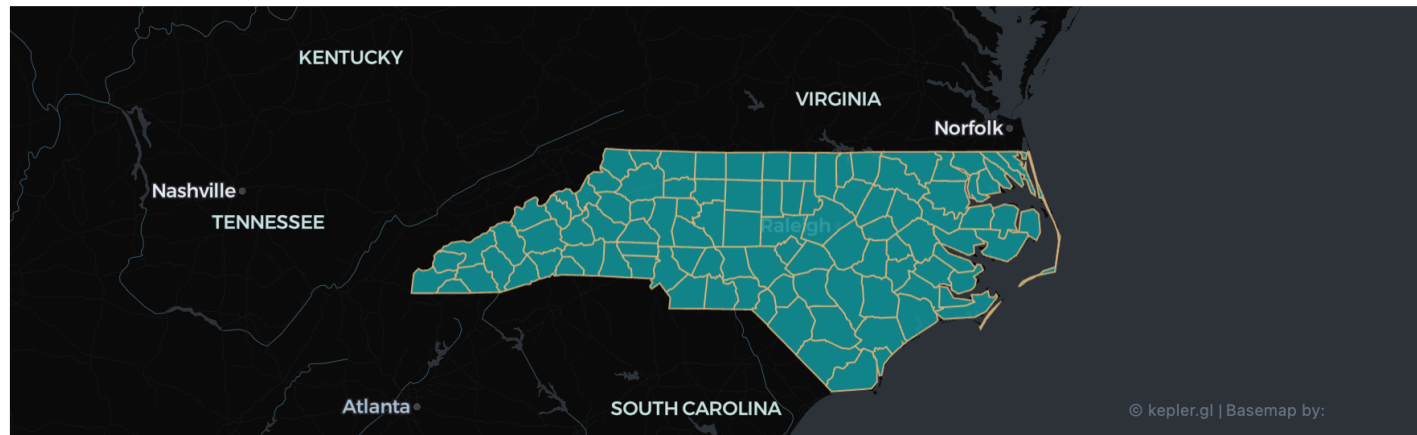
- ... which stores information on boundaries and shapes of geographic regions.
- Requires the `geopandas` library.

```
1 import geopandas as gpd
```



Choropleth Maps: `geom_map()`

SIDS Data



- We will use the SIDS data set from `geodatasets`.
- Details about the data available [here](#).

```
1 import geodatasets
2 path = geodatasets.get_path('geoda.sids')
3 df = gpd.read_file(path)
```



Choropleth Maps: geom_map()

SIDS Data

- The `geometry` column specifies the boundaries and shape of each county.
- This is the special column available in a `GeoDataFrame`.

```
1 df[['NAME', 'BIR79', 'geometry']].head()
```

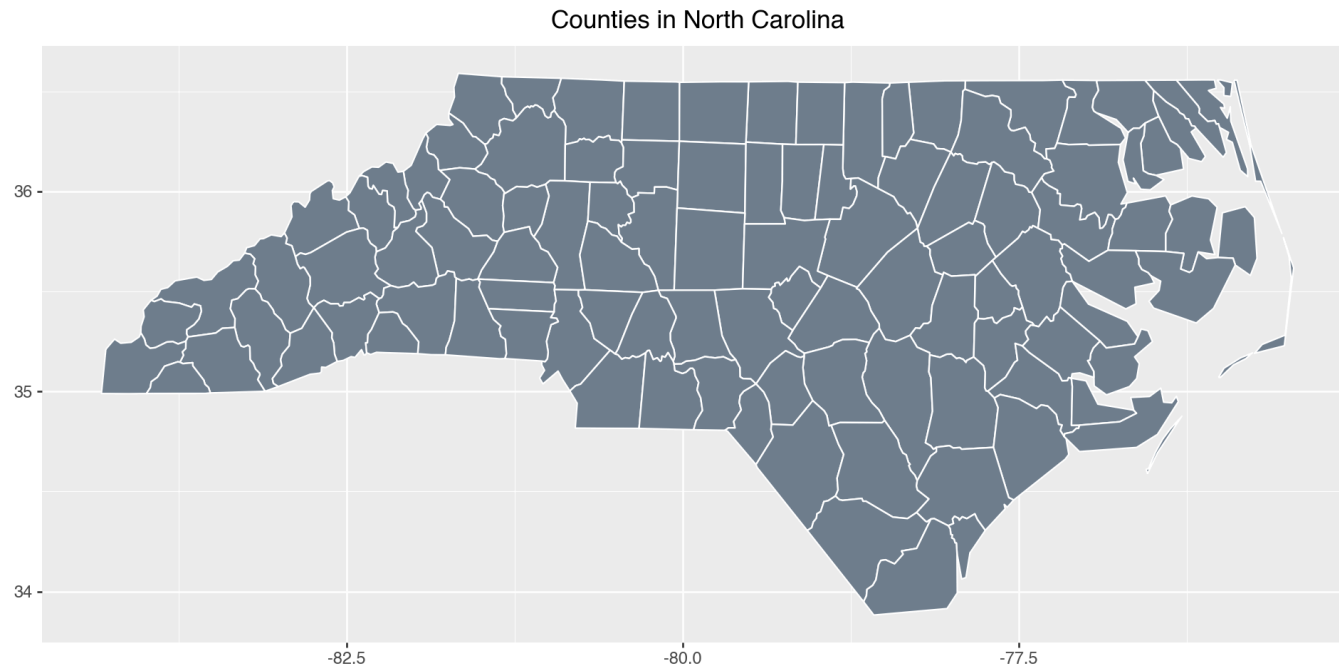
	NAME	BIR79	geometry
0	Ashe	1364.0	POLYGON ((-81.47276 36.23436, -81.54084 36.272...
1	Alleghany	542.0	POLYGON ((-81.23989 36.36536, -81.24069 36.379...
2	Surry	3616.0	POLYGON ((-80.45634 36.24256, -80.47639 36.254...
3	Currituck	830.0	MULTIPOLYGON (((-76.00897 36.3196, -76.01735 3...
4	Northampton	1606.0	POLYGON ((-77.21767 36.24098, -77.23461 36.214...

Choropleth Maps: `geom_map()`

SIDS Data: A Basic Map

`geom_map()` automatically detects the `geometry` column and plot the boundaries and shapes.

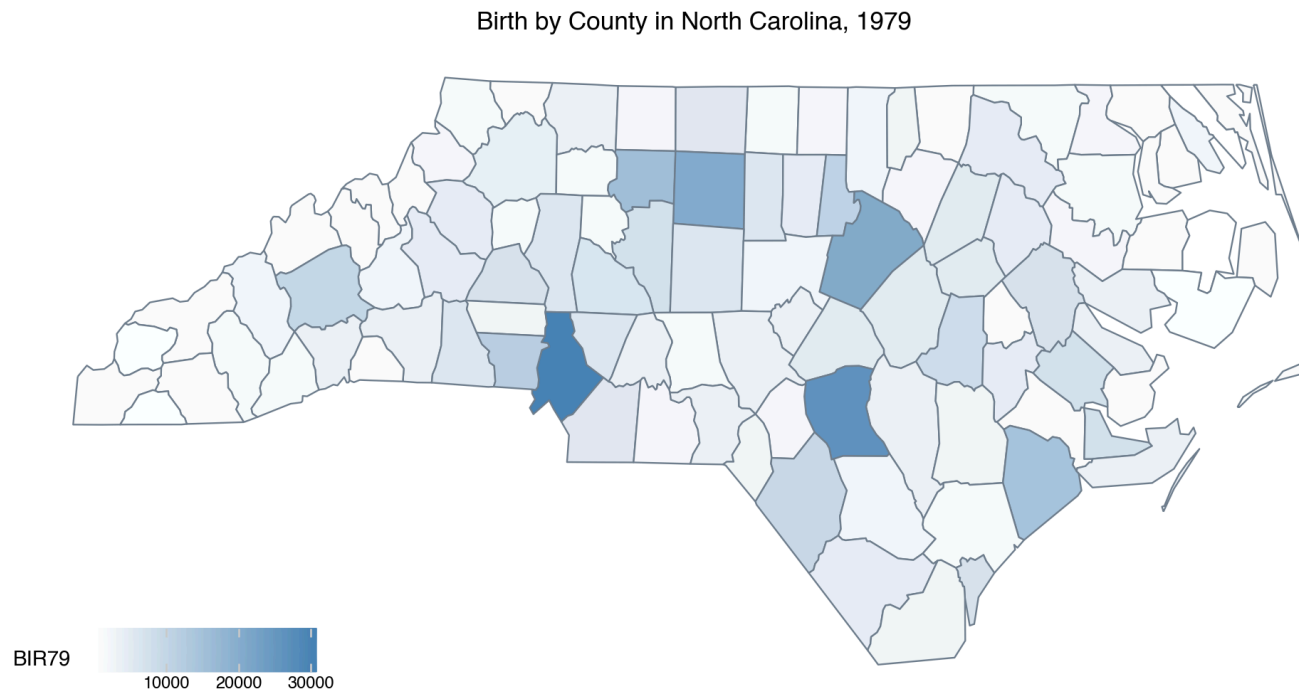
```
1 (ggplot(df) +  
2   geom_map(color = 'white', fill = 'slategray') +  
3   labs(title = 'Counties in North Carolina'))
```



Choropleth Maps: geom_map()

SIDS Data: A Choropleth Map

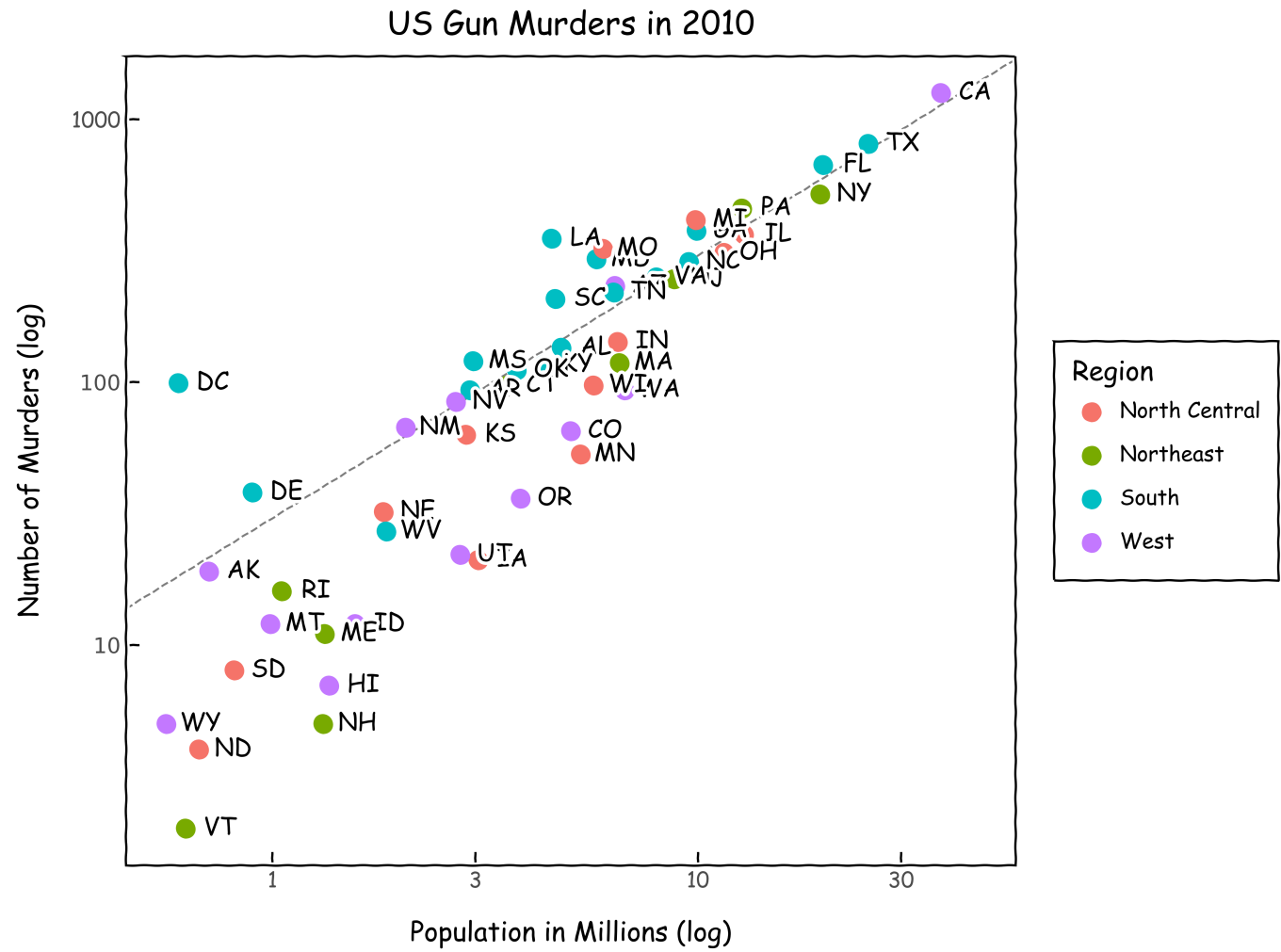
```
1 (ggplot(df) +  
2   geom_map(aes(fill = 'BIR79'), color = 'slategray') +  
3   scale_fill_gradient(low = 'white', high = 'steelblue') +  
4   labs(title = 'Birth by County in North Carolina, 1979') +  
5   theme_void() +  
6   theme(legend_position = (0, 0), legend_direction = 'horizontal'))
```



Case Study: US Gun Murders

US Gun Murders

Let us now recreate this graph we came across earlier.



US Gun Murders

We start by loading the data set, `wk11_murders.csv`.

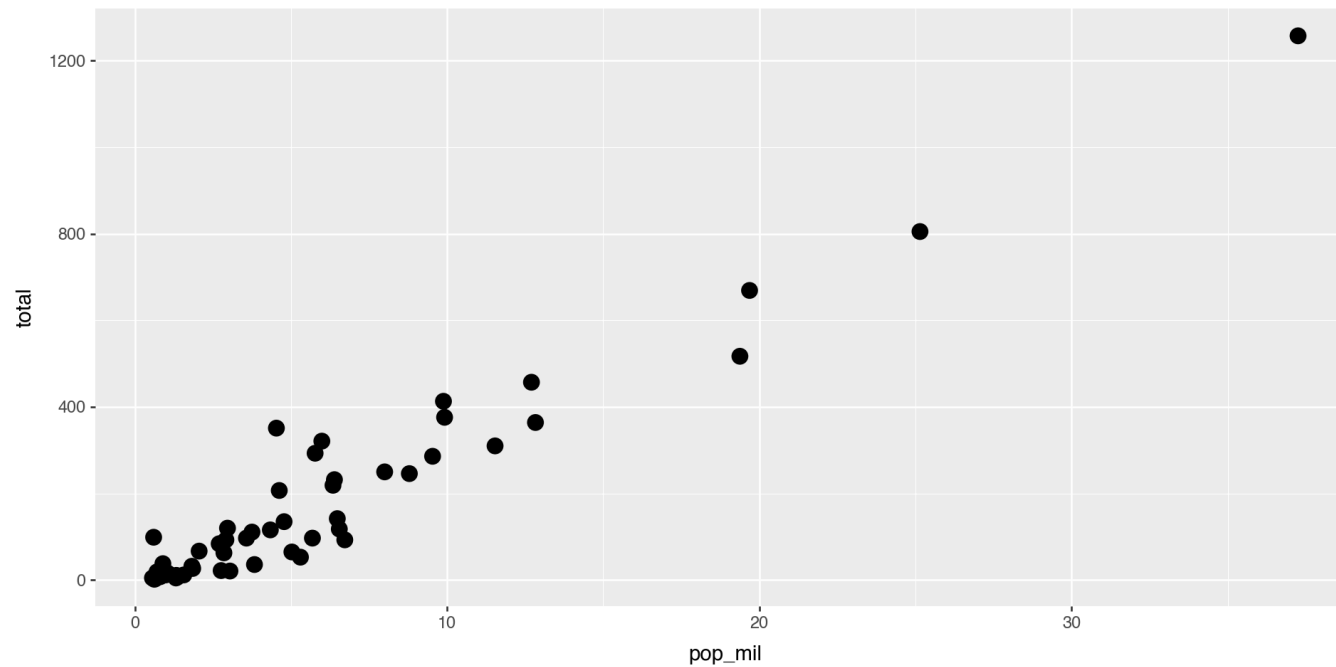
```
1 murders = pd.read_csv('../data/wk11_murders.csv')
2 murders['pop_mil'] = murders['population']/1e6
3 murders.head()
```

	state	abb	region	population	total	pop_mil
0	Alabama	AL	South	4779736	135	4.779736
1	Alaska	AK	West	710231	19	0.710231
2	Arizona	AZ	West	6392017	232	6.392017
3	Arkansas	AR	South	2915918	93	2.915918
4	California	CA	West	37253956	1257	37.253956

US Gun Murders

1. Aesthetics mappings.

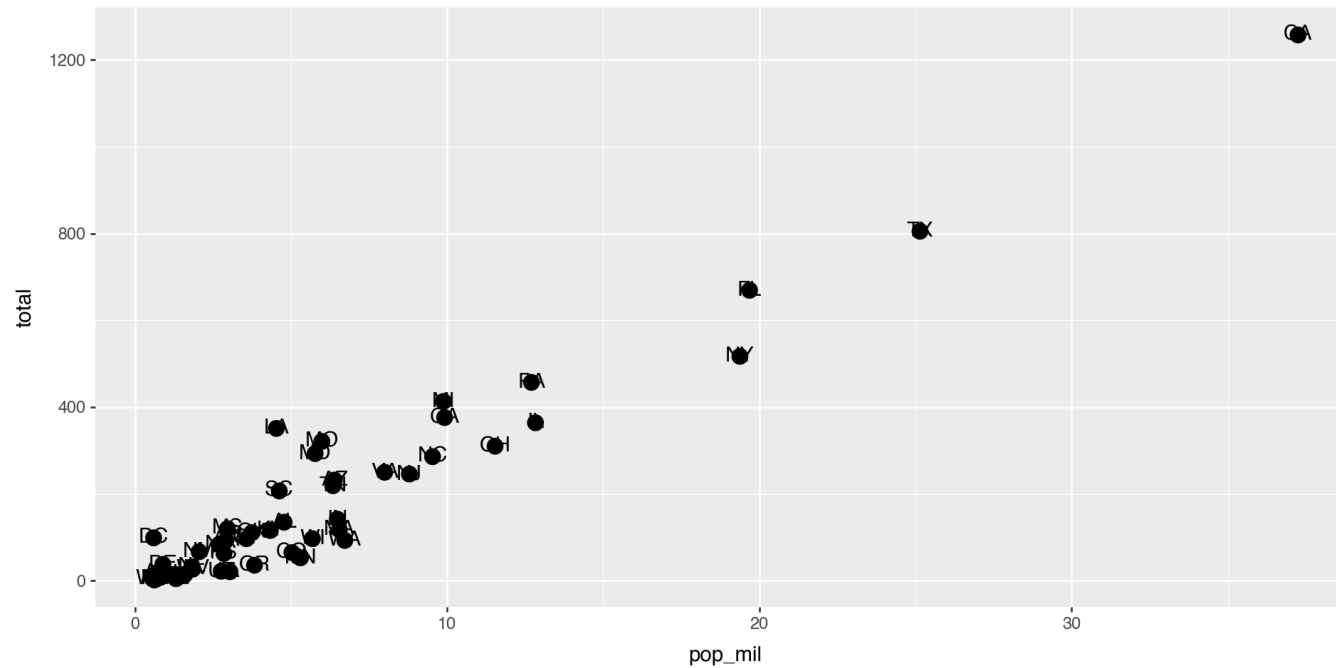
```
1 (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2   geom_point(size = 4))
```



US Gun Murders

2. Add labels to each point to identify the state.

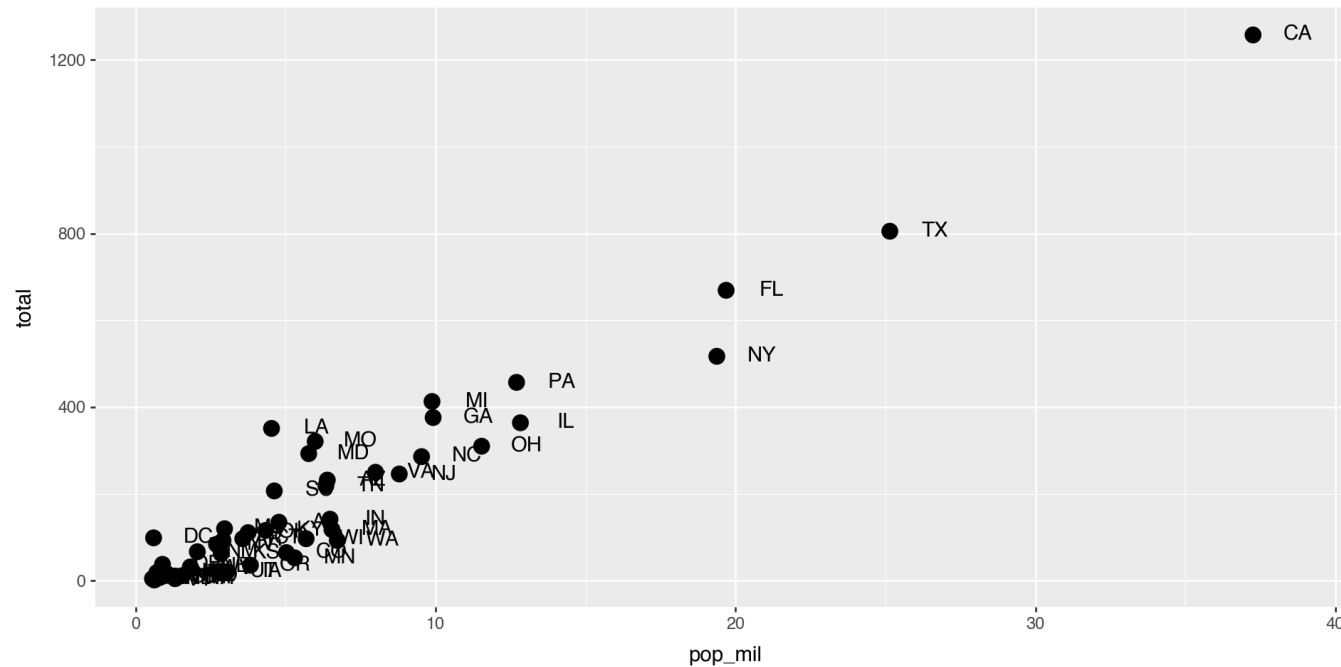
```
1 (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2   geom_point(size = 4) +  
3   geom_text(aes(label = 'abb')))
```



US Gun Murders

2. ... adjusting the positions of the labels.

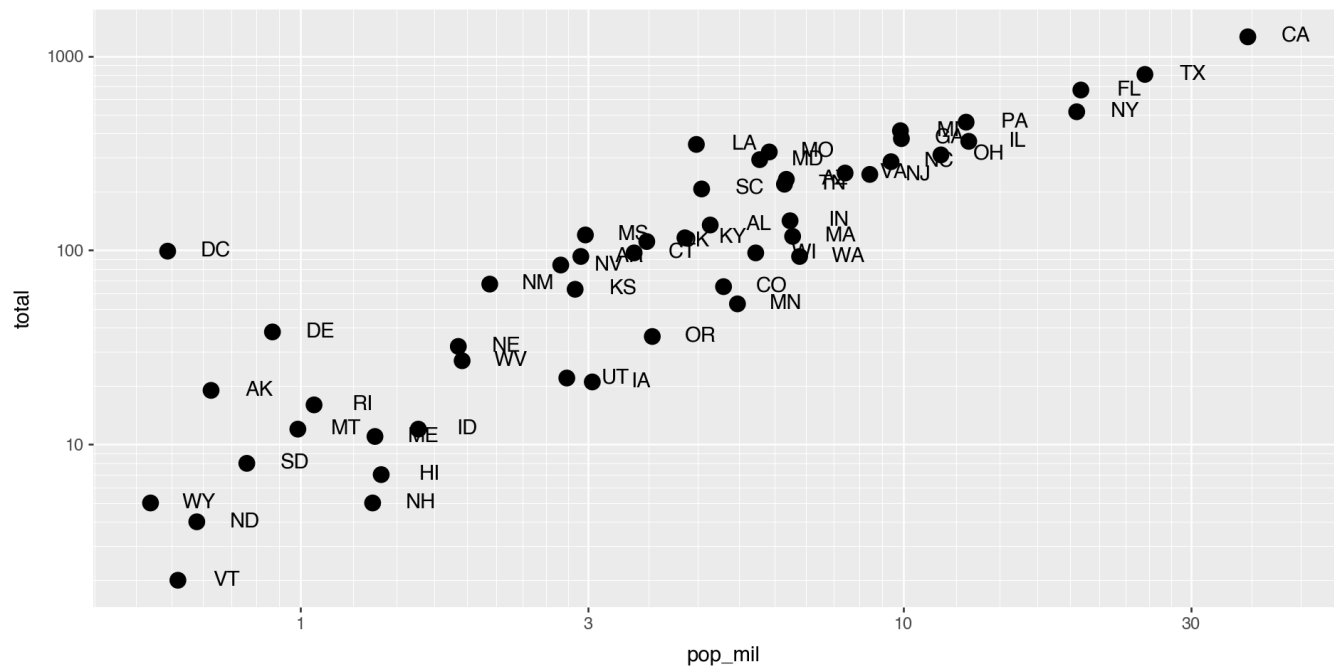
```
1 (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2   geom_point(size = 4) +  
3   geom_text(aes(label = 'abb'), nudge_x = 1.5))
```



US Gun Murders

3. Applying log-scale to both axes. `nudge_x` needs to be changed.

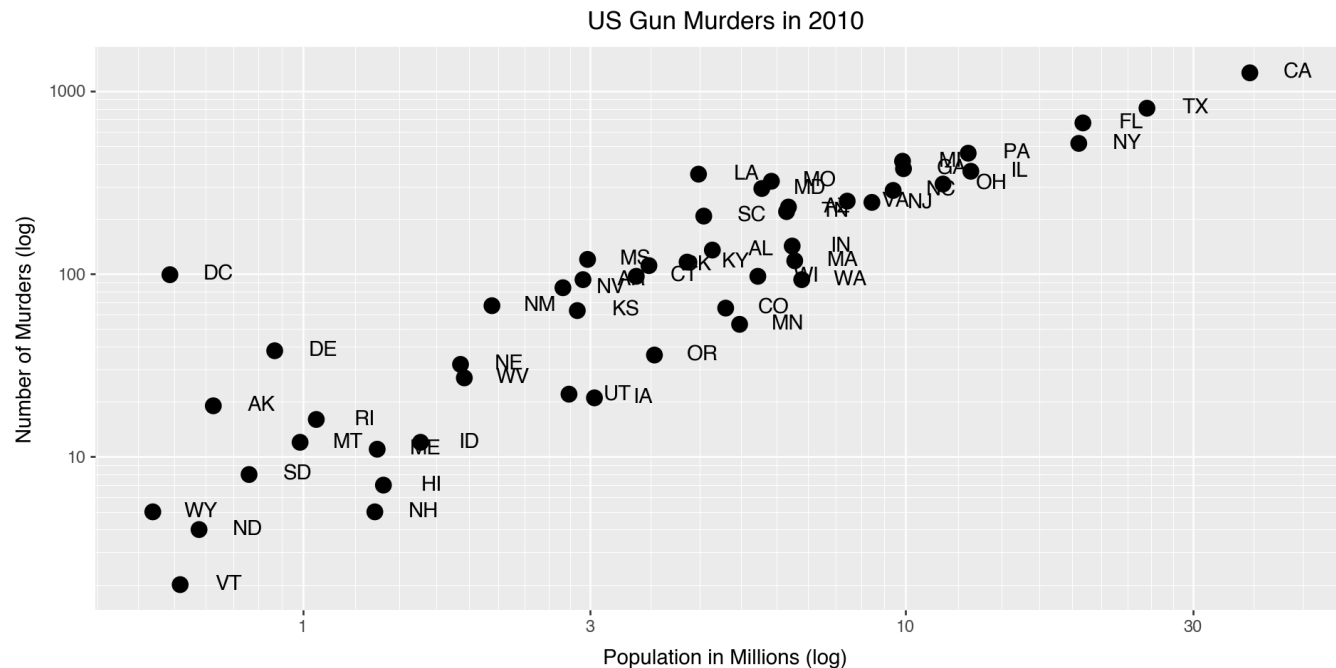
```
1 (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2   geom_point(size = 4) +  
3   geom_text(aes(label = 'abb'), nudge_x = 0.08) +  
4   scale_x_log10() + scale_y_log10())
```



US Gun Murders

4. Add axis labels and title.

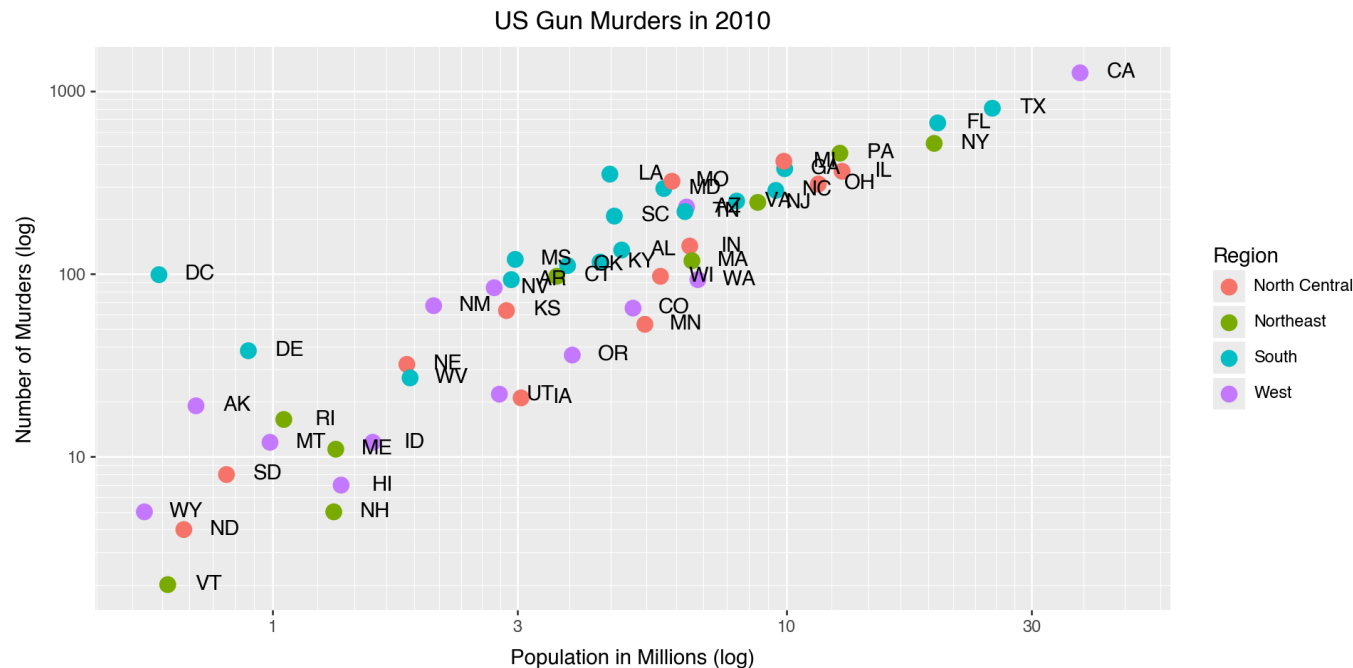
```
1 (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2   geom_point(size = 4) +  
3   geom_text(aes(label = 'abb'), nudge_x = 0.08) +  
4   scale_x_log10() + scale_y_log10() +  
5   labs(title = 'US Gun Murders in 2010',  
6         x = 'Population in Millions (log)',  
7         y = 'Number of Murders (log)'))
```



US Gun Murders

5. Different colours for different region of the states.

```
1 (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2   geom_point(aes(color = 'region'), size = 4) +  
3   geom_text(aes(label = 'abb'), nudge_x = 0.08) +  
4   scale_x_log10() + scale_y_log10() +  
5   labs(title = 'US Gun Murders in 2010',  
6         x = 'Population in Millions (log)',  
7         y = 'Number of Murders (log)', color = 'Region'))
```



US Gun Murders

6. Saving the plot as an object named `p`.

```
1 p = (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2     geom_point(aes(color = 'region'), size = 4) +  
3     geom_text(aes(label = 'abb'), nudge_x = 0.08) +  
4     scale_x_log10() + scale_y_log10() +  
5     labs(title = 'US Gun Murders in 2010',  
6           x = 'Population in Millions (log)',  
7           y = 'Number of Murders (log)', color = 'Region'))
```


US Gun Murders

7. Annotation, shapes, and adjustments.

- We next work on a dashed line that represents the average murder rate for the entire country.

$$\text{total_murder} = r \times \text{population}$$

- In log-scale, this becomes $\log(\text{total_murder}) = \log(r) + \log(\text{population})$.
- So on our plot, it is a line with slope 1 and intercept $\log(r)$.

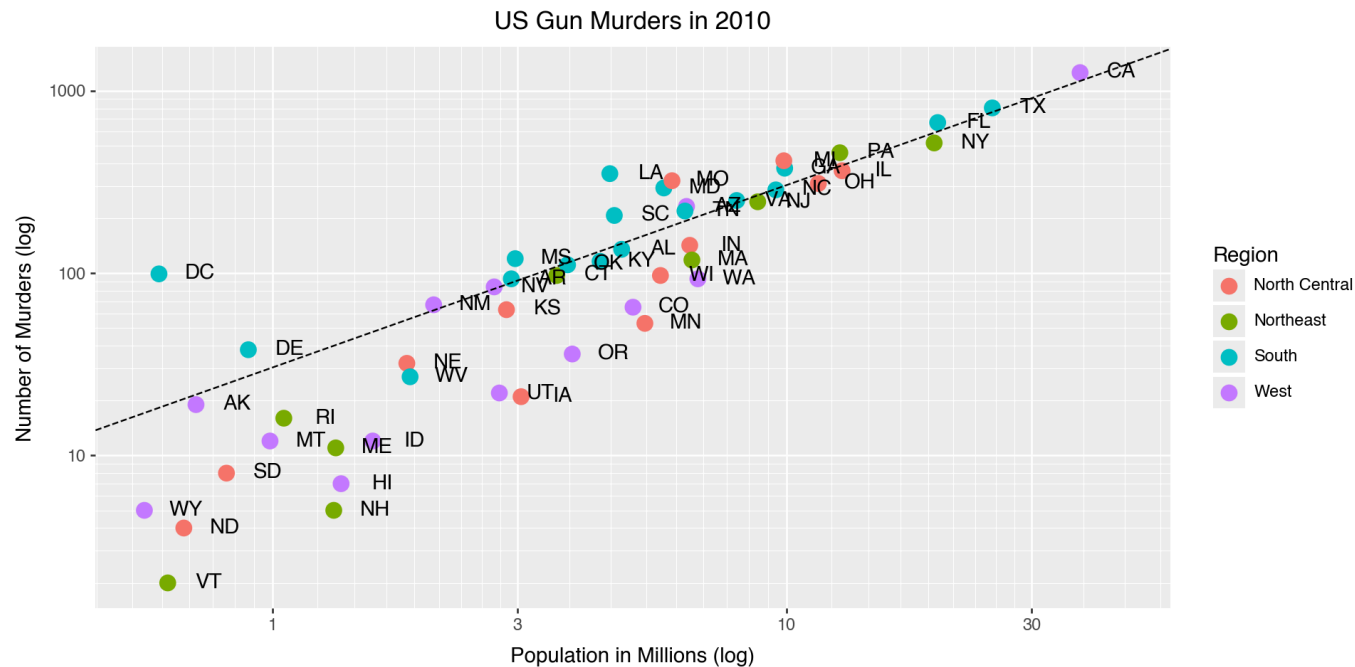
```
1 rate = murders['total'].sum() / (murders['population'].sum()/1e6)
2 rate = np.log10(rate)
3 print(rate)
```

1.4820949937153827

US Gun Murders

... now we add the line with `geom_abline()`.

```
1 p + geom_abline(slope = 1, intercept = rate, linetype = 'dashed')
```



US Gun Murders

8. Reordering Layers

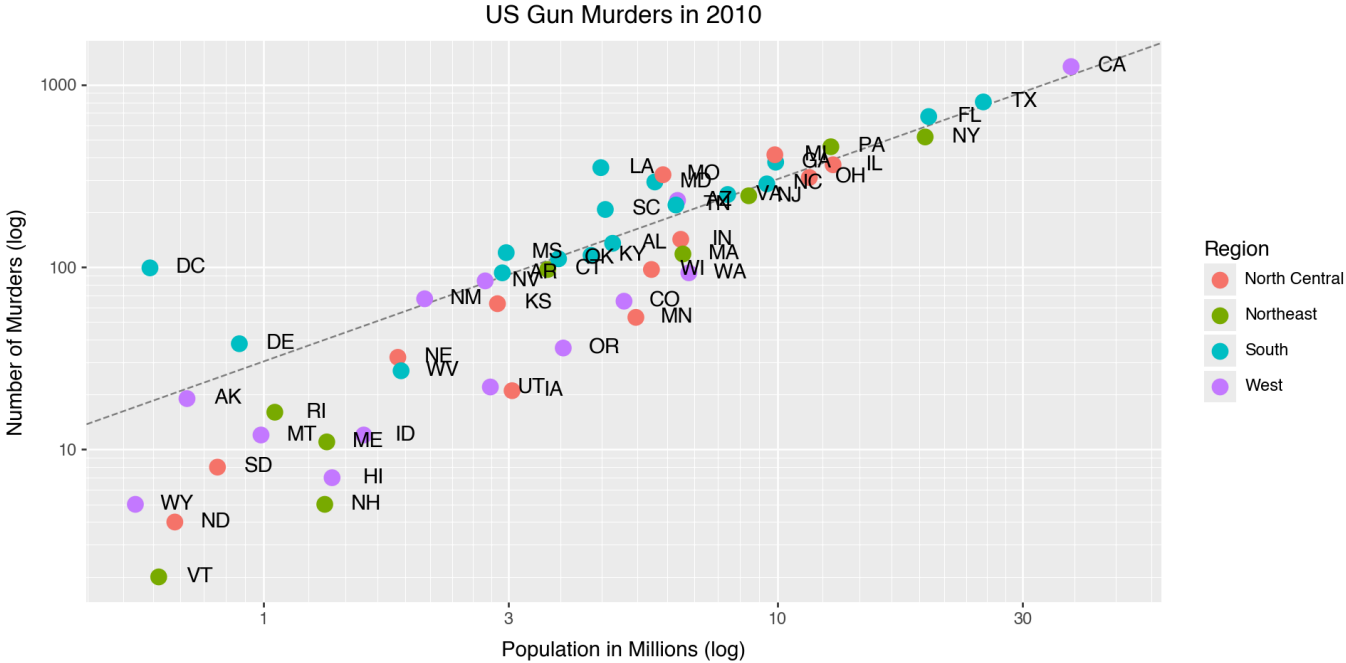
- Currently, the dashed line goes above the layer of points.
- We can reorder the layers by changing the order in which we use the `+` operator to add layers.

```
1 p1 = (ggplot(murders, aes(x = 'pop_mil', y = 'total')) +  
2       geom_abline(slope = 1, intercept = rate, linetype = 'dashed', color = 'gray'  
3       geom_point(aes(color = 'region'), size = 4) +  
4       geom_text(aes(label = 'abb'), nudge_x = 0.08) +  
5       scale_x_log10() + scale_y_log10() +  
6       labs(title = 'US Gun Murders in 2010',  
7           x = 'Population in Millions (log)',  
8           y = 'Number of Murders (log)', color = 'Region'))
```

US Gun Murders

... points above the line.

1 p1

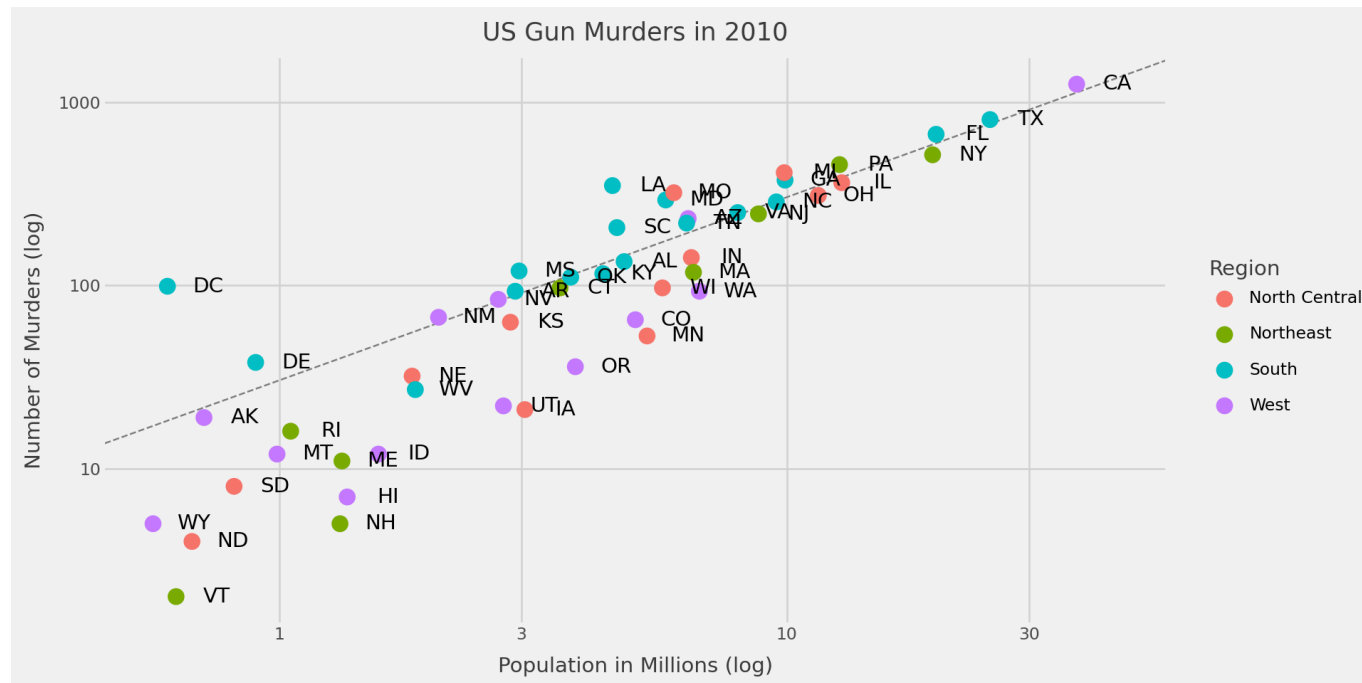


US Gun Murders

9. Alternative themes.

- We can apply a different theme to our plot.
- `theme_538()` is the same as the `fivethirtyeight` theme from `matplotlib`.

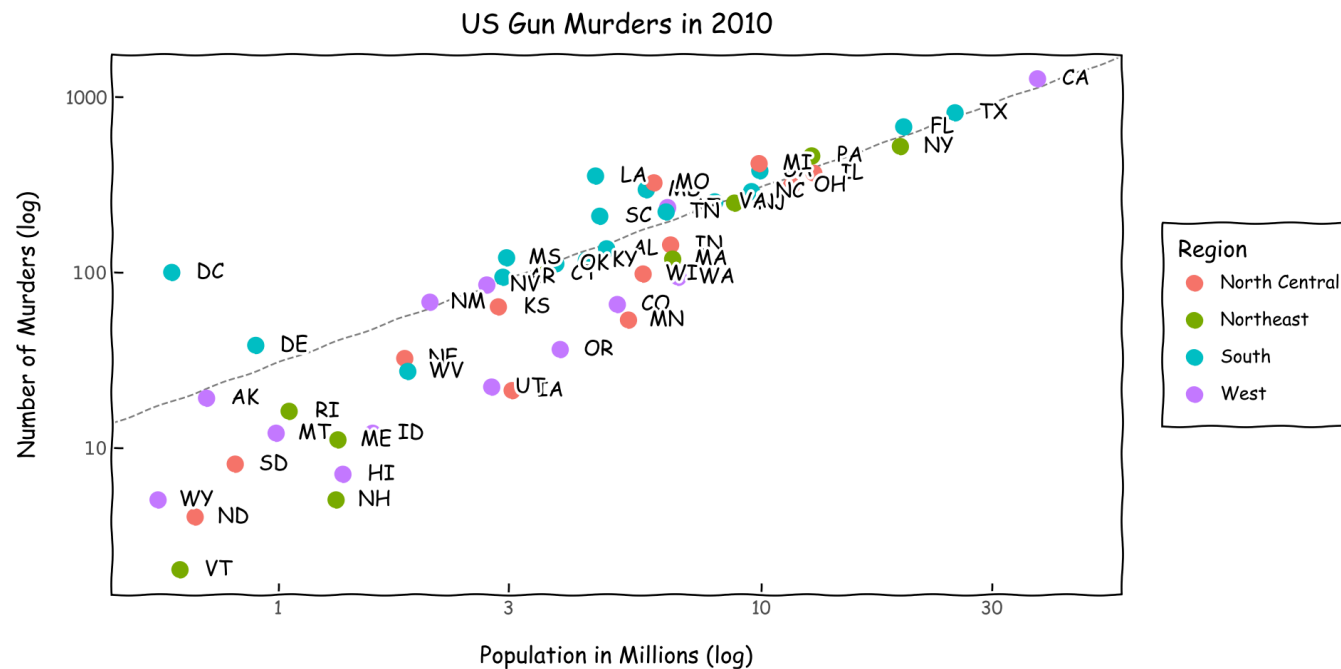
```
1 p1 + theme_538()
```



US Gun Murders

... now with the `xkcd` comic style theme.

```
1 p1 = p1 + theme_xkcd()  
2 p1
```



US Gun Murders

10. Lastly, we can export the plot as a portable network graph (.png) file with

- ... a width of 8 inches, height of 6 inches; and
- ... a resolution of 600 dots per inch (dpi).

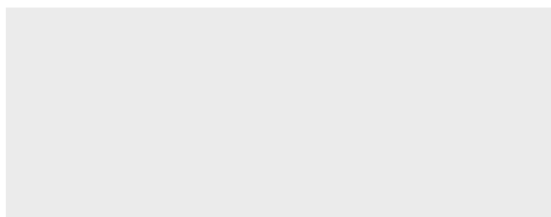
```
1 p1.save('US_Gun_Murders_xkcd.png', width = 8, height = 6, dpi = 600)
```



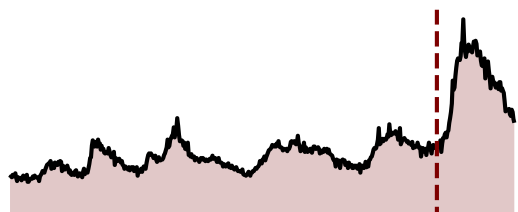
Summary

Here are some `geom` functions we have explored this week.

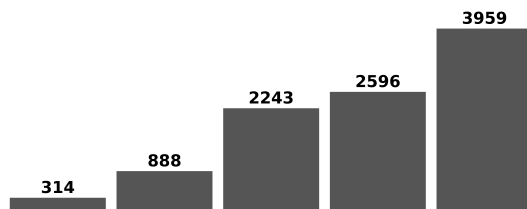
`ggplot()`



`+ geom_line()` `+ geom_area()`
`+ geom_vline()`



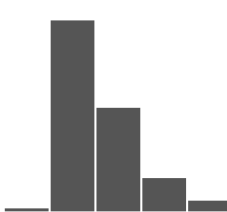
`+ geom_bar()` `+ geom_text()`



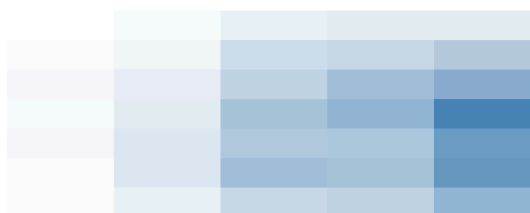
`+ geom_point()`



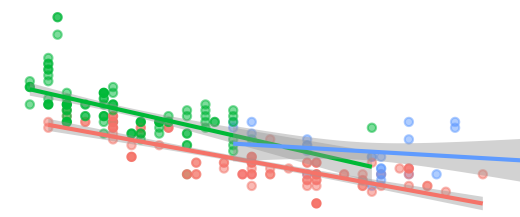
`+ geom_histogram()`



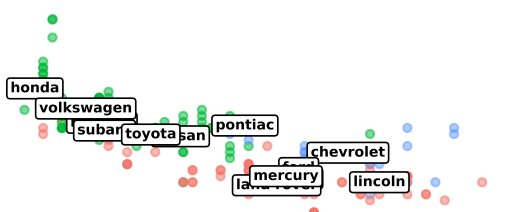
`+ geom_tile()`



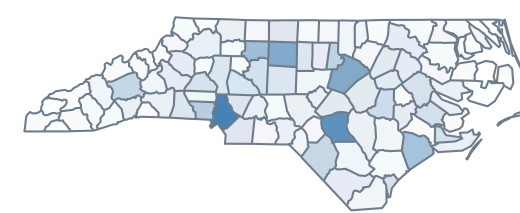
`+ geom_point()` `+ geom_smooth()`



`+ geom_point()` `+ geom_label()`



`+ geom_map()`



Summary: Layered Grammar of Graphics

Our initial template can be extended to:

```
1 (ggplot(<DATA>) +  
2   <GEOM_FUNCTION>(aes(<MAPPINGS>),  
3     position = <POSITION>) +  
4   <COORDINATE_FUNCTION> +  
5   <FACET_FUNCTION> +  
6   <THEME_FUNCTION>)
```

- These parameters in the template compose a formal grammar of graphics.
- Any plot can be described as a combination of these parameters.

Summary: plotnine themes

plotnine provides several themes that controls the non-data elements of the plot.

